



EmulatR User and Developer Reference

© 2026 Timothy Peer - eNVy Systems Inc.

Table of Contents

Foreword	0
Part I Introduction	6
1 What is EmulatR	6
2 License	7
3 Goals & Objectives	8
4 Supported Systems	9
5 Current Platform Support	10
6 Feature Matrix	10
7 Current Development Status	11
Part II Installation	13
1 Required Tools & Frameworks	13
2 Building from Source	14
3 Firmware Requirements	14
4 First Startup	15
Part III Quick Start	16
1 Launching EmulatR	16
2 Loading Firmware	17
3 Running from the shell	17
4 Connecting to the Console	21
5 Reaching the SRM Prompt	21
6 Clean Shutdown	22
Part IV Operating EmulatR	23
1 Configuring & Running	23
Configuring	24
Executing	25
Executing Two Instances	25
Example - Performance	26
2 Console Connectivity	28
Console Support	28
TCP Console Overview	29
PuTTY / Plink / TeraTerm / MobaXterm / Netcat	29
3 SRM Console	30
SRM Overview	31
Common Commands	31
show config	32
show device	32
show memory	32

boot	33
set	33
SRM Environment Variables	33
4 Storage	34
Create Guest Disk Storage	34
Virtual Disk Images	36
Virtual CDROM	36
IDE Devices	37
ATAPI Devices	37
ISO-9660 PFD (CD001)	37
SCSI Devices	38
Storage Interconnect and Fabric	38
SDI Devices / DSSI Devices / ST506 Devices	39
5 Snapshots	39
Automatic / Creating / Restoring / Pruning	40
6 Firmware Update Utility (LFU)	40
List / Display / Update	41
7 Guest Boot	42
8 Logging & Diagnostics	43
SRM Performance Logging	44
Diagnostics & Debugging	44

Part V Architecture & Internals 46

1 Overview	46
2 Memory	50
3 Addressing	50
4 Firmware Decode & Decompression	51
5 Execution Flight Plan	52
6 Clock Frequency MHz	53

Part VI Reference 55

1 Environment Variables (EMULATR_*)	55
2 Glossary	57
3 Dependencies	58
4 Attributions & Credits	59

Part VII Troubleshooting 61

1 EmulatR will not start	61
2 Firmware Hangs / Slow Boot / Missing Devices	62
3 Console Connection Issues	62
4 Halt Button is IN, BOOT NOT POSSIBLE	63
5 Example (Console) execution	63
6 >>> Evaluate Address	64

Part VIII Appendices - Developer Reference 66

Part IX Platform Appendices	67
1 Appendix - DS10	67
2 Env Vars Operation Ref / Kill the EmulatR / Known Limitations /	68
3 SRM Boot / SRM Commands Prompt / SuperIO Controller /	69
4 Task Matrix / TimeOut Loops	70
5 Appendix DS20 / ES40 / ES45	72
Part X Subsystem Internal	73
1 Floating-Point Considerations	73
2 grainFactoryLib	75
3 DispatchTables	75
4 CMake Macro Guards	76
Part XI Architecture Deep-Dives	79
1 Introduction & Overview	79
2 Memory Map & IPRs	83
3 Addressing & TLB	85
4 Decompressor Internals	87
5 Execution / Boot Internals	89
Part XII Tools	91
1 Ghidra	91
2 Help & Manual	92
3 Software Verify	92
4 Qt Creator	93
5 Host Decompressor	93
Index	97

1 Introduction

EmulatR is a software emulator of the Digital (DEC) Alpha AXP family. It models the 21264 / 21264A (EV6 / EV67) processor and the 21272 "Tsunami / Typhoon" core-logic chipset, and is complete enough to load and run original, unmodified DEC SRM console firmware to the interactive >>> prompt.

This guide serves two audiences. Operators will find everything needed to build EmulatR, load firmware, connect a terminal, drive the SRM console, and shut down cleanly. Developers will find the architecture and subsystem internals in the later sections. Where a fact is not yet confirmed against the current build, the text flags it with a [TODO] marker rather than guessing.

In this section

What is EmulatR -- the emulated machine at a glance.

Supported Systems and Current Platform Support -- the DS10 target and the ES40 / ES45 placeholders.

Feature Matrix -- per-subsystem status (working, in progress, planned).

Current Development Status -- where the project stands right now and what is next.

Where to go next

New here? Start with Quick Start for the fastest path from a built binary to the >>> prompt. To build EmulatR first, see Installation. Once at the prompt, the SRM Console section covers the commands; Troubleshooting covers what to check when something does not start.

See Also.

[What is EmulatR](#)  6

[Supported Systems](#)  9

[Current Platform Support](#)  10

[Feature Matrix](#)  10

[Current Development Status](#)  11

1.1 What is EmulatR

EmulatR is a software emulator of the Digital (DEC) Alpha AXP family. It models the 21264 / 21264A (EV6 / EV67) processor and the 21272 "Tsunami / Typhoon" core-logic chipset, targeting AlphaServer DS10 / ES40 / ES45-class machines.

Its goal is faithful, cycle-aware emulation -- complete enough to load and execute original, unmodified DEC SRM console firmware and reach the interactive >>> prompt. The core is

written in modern C++ (C++20) and is kept free of any GUI-framework dependency; the console subsystem layers on top of it.

At a glance

CPU: Alpha 21264A (EV6 / EV67), with OpenVMS and Tru64 UNIX PALcode.

Core logic: 21272 Tsunami / Typhoon -- Cchip (21272-CA), Dchip (21272-DA), Pchip (21272-EA).

Southbridge: Cypress CY82C693 -- PCI-to-ISA bridge, IDE/ATAPI, and SuperIO (FDC, COM1/COM2, LPT1).

Firmware: runs a real SRM image, for example DS10 SRM V7.3 (ds10_v7_3.exe).

Console: a RAW TCP server, reached with any terminal client (PuTTY, plink, and similar).

See Also.

[Supported Systems](#) 

[Current Platform Support](#) 

[Feature Matrix](#) 

[Introduction](#) 

1.2 License

ASA-EMulatR

Copyright © 2025–2026 Timothy Peer (eNVy Systems, Inc.)

<https://envysys.com>

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Commercial Licensing

This project is also available under a separate commercial license. If you wish to use ASA-EMulatR in a proprietary product, closed-source environment, or in a manner incompatible with the GNU General Public License, you must obtain a commercial license from:

Timothy Peer
eNVy Systems, Inc.
peert_at_envysys_dot_com
<https://envysys.com>

Commercial licensing terms are available upon request.
For the full license terms and commercial-licensing inquiries, contact Timothy Peer / eNVy Systems, Inc. at peert_at_envysys_dot_com.

See Also.

[Attributions & Credits](#)  59

1.3 Goals & Objectives

EmulatR is a full-system, cycle-based, high-fidelity emulator of the DEC Alpha AXP architecture. Its purpose is not to approximate an Alpha but to behave like one -- faithfully enough that the most demanding original software cannot tell the difference. This page sets out the single standard the project is held to and the priority-ordered goals every design decision serves.

The fidelity standard

EmulatR is governed by one overriding standard: it must behave like real Alpha hardware in every architecturally visible way, even when doing so complicates the implementation. This is applied uniformly across all subsystems -- the instruction set, memory translation, cache and coherency behavior, the PAL boundary, interrupt and exception delivery, device and DMA semantics, and multiprocessor ordering. A subsystem is correct only when no architecturally visible difference from hardware can be observed through it.

The test of fidelity is black-box and adversarial. The judges are not internal assertions but the most demanding guests available: the SRM console and PALcode exercise the privileged architecture directly, a booting operating system exercises the memory model, trap machinery, and device fabric under real workloads, and differential validation against a reference emulator checks behavior at the instruction and subsystem level. Fidelity here means architectural-semantic accuracy, not the reproduction of a particular chip's timing: EmulatR reproduces exactly what the architecture makes visible -- the cycle-based run loop, the precise point at which results and exceptions commit, the ordering guarantees and barriers, and the privileged state transitions.

Primary goals (in strict priority order)

EmulatR is designed around five primary goals. Every design decision is evaluated against them in this order; when two goals conflict, the higher-priority goal wins.

1. **Architectural correctness first.** The emulator models Alpha's architectural semantics, not its performance characteristics. Correctness is always prioritized over speed.
2. **Deterministic and debuggable execution.** Execution must be understandable, reproducible, and inspectable. One iteration of the run loop corresponds to one hardware clock cycle, enabling deterministic replay and debugging.
3. **Explicit serialization.** Ordering and synchronization are never implicit. All serialization is driven by architectural mechanisms -- the MB, WMB, EXCB, and TRAPB barriers, PAL entry, and exceptions -- and exceptions are delivered precisely at the architectural commit point, the Writeback stage.
4. **SMP as a first-class concern.** Multiprocessor behavior is fundamental to the design, not layered on afterward. Cross-CPU coordination is modeled with explicit barriers, reservation tracking, and IPI delivery.
5. **Separation of concerns.** Instruction decode, execution, memory access, coherency, privilege, and devices are isolated into well-defined functional domains (the "Boxes") with one-way dependency flow. No domain reaches into another's internal state.

Strategy: one set of objectives, four implementations

These objectives have been invariant across the entire history of the project. Four distinct implementations have been written; each changed the mechanism, never the goals. The current design (V4) is the first to satisfy all five at once -- it keeps a flat, shallow dispatch path for determinism and debuggability while resolving each instruction to a faithful, fully-architected execution leaf. The strategy throughout has been to let the implementation evolve toward the fixed standard, never to relax the standard to fit an implementation.

See Also.

[What is EmulatR](#) 

[Architecture Overview](#) 

[Current Development Status](#) 

1.4 Supported Systems

EmulatR targets the Tsunami / Typhoon AlphaServer line. The emulated hardware reported by the SRM console (`show config`) reflects a DS10-class machine:

- *Processor: Alpha EV6 pass 1, 268 MHz (a calibrated figure -- see Clock Frequency and the MHz Reading), Bcache disabled.*
- *Core logic: DECchip 21272-CA (Cchip) Rev 2, 21272-DA (Dchip) Rev 1, 21272-EA (Pchip 0) Rev 1; TIG / Arbiter Rev 7.31.*

- *Memory: 1024 MB default array (configurable via [System] memorySize in the ini or --mem), base 0x00000000.*
- *I/O: PCI Hose 00 with the Cypress 82C693 bridge / IDE at bus 0, slot 5; ISA devices MOUSE, KBD, COM1, COM2, LPT1, FLOPPY.*

Platform selection is driven by a per-model bus-configuration manifest (see Configuring). DS10 is the actively developed and validated platform; ES40 and ES45 manifests are scaffolded (see Current Platform Support).

See Also.

[What is EmulatR](#) 

[Current Platform Support](#) 

[Configuring](#) 

1.5 Current Platform Support

A platform manifest is selected from the model name as lower(model) + "_platform.json" (for example, DS10 gives ds10_platform.json). The table below summarizes current status. "Validated" means the platform has been driven to the SRM >>> prompt in EmulatR.

Platform	Manifest	CPU / Chipset	Status
DS10	ds10_platform.json	EV6 / 21272	Active -- boots to >>>
ES40	es40_platform.json	EV6 / 21272	Manifest present -- not yet validated
ES45	es45_platform.json	EV67 / 21272	Manifest present -- not yet validated

A missing platform file is a hard error -- there is no silent fallback. The selection order is: EMULATR_PLATFORM_CONFIG, then a platformConfigPath ini key, then the derived name.

See Also.

[Supported Systems](#) 

[Feature Matrix](#) 

[Configuring](#) 

1.6 Feature Matrix

Subsystem status as of June 2026. This reflects the DS10 target; status is expected to evolve and should be reconfirmed against the current build.

Subsystem	Status	Notes
CPU pipeline (EV6) + TLB	Working	128 ITB + 128 DTB; PALcode relocation in place.
SRM firmware boot to >>>	Working	Reaches the interactive console prompt.
TCP console server	Working	RAW TCP; PuTTY auto-launch.
Snapshot save / restore	Working	.axpsnap; autoload-newest, marker capture.
SuperIO (FDC / COM / LPT)	Working	CY82C693 SuperIO word-fold confirmed.
IDE / ATAPI virtual CD-ROM	In progress	dqa0 enumeration gap under investigation.
IIC (PCF8584)	Under investigation	S1 returns 0x01 where firmware expects 0x00.
Flash / NVRAM persistence	Under investigation	"Flash ROM writes are disabled" -- see Known Limitations.
boot / OS load (auto_action BOOT)	Planned	HALT is the only fully implemented action.
SCSI	Planned	[TODO: confirm -- no SCSI HBA currently modeled.]

See Also.
[Current Development Status](#) ¹¹
[Current Platform Support](#) ¹⁰

1.7 Current Development Status

This page is a plain-language snapshot of where EmulatR stands as of June 2026 -- what works end to end, the milestone the project is working toward next, and the known rough edges. It is deliberately a moving target; the authoritative day-to-day record lives in the engineering journals and the dated CHECKPOINT notes. For a per-subsystem breakdown see the Feature Matrix.

What works today

A clean cold boot runs the original, unmodified DEC SRM console firmware all the way to the interactive >>> prompt, fully usable over a serial terminal. The commands `show config`, `show memory`, `show pal` and `show version` all return correct output. Firmware is a real SRM 7.3-1 image carried in emulated flash; the LFU `update srm` path works, and machine-state snapshots have been captured for both the firmware-update (`UPD>`) and console (`>>>`) states.

The current frontier: getting to an OS boot

The next major milestone is booting an operating system. Two concrete gaps stand between the prompt and that goal:

No bootable disk is enumerated yet. At the prompt, show device lists only the floppy (dva0) -- there is no dka SCSI disk to boot from. Disk-image support and the storage backing exist (see Virtual Disk Images); the gap is the device model and enumeration that would make a disk appear to the firmware.

On-board PCI devices are not walked. show config reports only the Cypress 82C693 bridge on PCI Hose 00 -- the on-board Ethernet and SCSI controllers are not yet modeled or enumerated, so the firmware cannot find them.

A separate active investigation concerns the front-panel halt gate: `b dqa1` currently reports "BOOT NOT POSSIBLE". This is being diagnosed with an armed hardware trace to identify the exact source the firmware tests before any fix is written -- an evidence-first approach rather than guessing at a register.

Known cosmetic issues

Two harmless display quirks are tracked: `show device` prints each line twice (console line-doubling, also seen at `UPD>`), and the SROM revision shown by `display / show config` contains one garbled byte. Neither affects operation.

What is planned next

- *SRM environment persistence -- make set at the prompt survive a reset and cold boot, like real NVRAM.*
- *PCI bus enumeration with on-board device models (Ethernet, SCSI), so the firmware discovers real hardware instead of an all-ones placeholder.*
- *A SCSI disk plus boot media so a dka device exists to boot, leading into an OS install from CD-ROM.*

Snapshot date: June 2026. Because status changes frequently, treat this page as orientation and confirm specifics against the current build and the latest CHECKPOINT journal.

See Also.

[Feature Matrix](#)  10

[Architecture Overview](#)  46

[Halt Button is IN, BOOT NOT POSSIBLE](#)  63

2 Installation

This chapter covers obtaining the tools, building EmulatR from source, supplying an SRM firmware image, and the first launch. Build specifics are marked with [TODO] where they still need to be confirmed against the current project files.

In this section

- *Required Tools and Frameworks -- the compiler, build system, and libraries.*
- *Building from Source -- configuring and building the binary.*
- *Firmware Requirements -- supplying the SRM console image.*
- *First Startup -- the first launch through to the prompt.*

See Also.

[Required Tools and Frameworks](#) ¹³

[Building from Source](#) ¹⁴

[Firmware Requirements](#) ¹⁴

[First Startup](#) ¹⁵

[Quick Start](#) ¹⁶

2.1 Required Tools & Frameworks

Tooling and libraries used by EmulatR. Exact version requirements are to be confirmed.

- *Compiler: MSVC with C++20 support. QtCreator, [TODO: confirm minimum toolset version.]*
- *Build system: CMake. [TODO: confirm minimum version.]*
- *Qt: used by the console / UI subsystem only -- the emulator core is deliberately Qt-free. [TODO: confirm version.]*
- *spdlog: logging infrastructure (dedicated EMULATR_CONSOLE_TRACE channel). [TODO: confirm version.]*
- *A terminal client for the console: PuTTY / plink (auto-launched), or any RAW-TCP client.*
- *Optional: Ghidra, for SRM firmware reverse-engineering work.*

See Also.

[Building from Source](#) ¹⁴

[Dependencies](#) ⁵⁸

[Ghidra](#) ⁹¹

2.2 Building from Source

EmulatR is a CMake project built with MSVC (C++20). The build tree produces `Emulatr.exe` under `out/build/<config>/`, with the two configurations in active use being `release` and `relwithdebinfo` (the latter is the day-to-day debug / profiling build).

General shape of a build (exact preset and generator names to be confirmed):

```
# configure
cmake --preset <preset> # [TODO: confirm preset name]

# build (RelWithDebInfo is the standard working build)
cmake --build --preset <preset> --config RelWithDebInfo

# resulting binary
./out/build/relwithdebinfo/Emulatr.exe
```

[TODO: confirm and document the exact configure / build commands, the CMake generator, and any dependency provisioning (for example Qt and spdlog acquisition) for a clean checkout.]

Note: the diagnostic build flag `EMULATR_MEMDIAG` is a CMake option, not a runtime variable, and must stay OFF for normal use -- its per-instruction logging imposes roughly a 900x slowdown.

See Also.

[Required Tools & Frameworks](#) ¹³

[Firmware Requirements](#) ¹⁴

[First Startup](#) ¹⁵

2.3 Firmware Requirements

EmulatR requires an SRM console firmware image to execute. The DS10 reference image is an SRM V7.3 build (`ds10_v7_3.exe`).

The image is selected in precedence order:

*--firmware <path> on the command line (overrides the ini for a single run); or
the `[ROM]/firmwareImage` ini key (default: `ds10_v7_3.exe`).*

Path gotcha: the ini value is the bare file name (no `firmware/` prefix) and resolves against the current working directory. Launch from the Emulatr root, or pass an explicit `--firmware firmware/ds10_v7_3.exe`.

Optional integrity check: set `[ROM]/firmwareSha256` to verify the image at load. When empty, the integrity check is skipped.

[TODO: document firmware provenance, licensing, and the set of supported SRM versions.]

See Also.

[First Startup](#) 

[Loading Firmware](#) 

[Building from Source](#) 

2.4 First Startup

A first run, launched from the EmulatR root so relative paths resolve correctly:

```
cd /d/EmulatR/EmulatRAppUniv4/EmulatR
./out/build/relwithdebinf/EmulatR.exe --firmware firmware/ds10_v7_3.exe
```

On launch the TCP console server comes up (default port 10023) and, unless `EMULATR_NO_PUTTY` is set, a PuTTY session is auto-launched against it. The firmware then runs through its power-up sequence to the SRM >>> prompt. For the step-by-step walkthrough, see Quick Start; to stop cleanly, see Clean Shutdown.

See Also.

[Quick Start](#) 

[Launching EmulatR](#) 

[Clean Shutdown](#) 

[**Running from the shell**](#) 

3 Quick Start

The fastest path from a built binary to an interactive SRM prompt, and back down cleanly. Each step has its own topic below; follow them in order.

In this section

- *Launching EmulatR -- start the emulator from the EmulatR root.*
- *Connecting to the Console -- attach a terminal to the TCP console.*
- *Loading Firmware -- point the run at the SRM image.*
- *Reaching the SRM Prompt -- what the power-up sequence shows.*
- *Clean Shutdown -- stop the emulator so NVRAM is flushed.*

See Also.

[Launching EmulatR](#) ¹⁶

[Connecting to the Console](#) ²¹

[Loading Firmware](#) ¹⁷

[Reaching the SRM Prompt](#) ²¹

[Clean Shutdown](#) ²²

3.1 Launching EmulatR

Launch from the EmulatR root. A genuine cold boot (no snapshot restore), quiet, is:

```
cd /d/EmulatR/EmulatRAppUniv4/EmulatR
EMULATR_NO_AUTOLOAD=1 ./out/build/relwithdebinfo/EmulatR.exe \
  --firmware firmware/ds10_v7_3.exe --autosnapshot off 2> run.log
```

Omit `EMULATR_NO_AUTOLOAD` to restore the newest snapshot instead of cold-booting. Diagnostics are written to the redirected stderr (`run.log`).

Common launch modes

Autoload to the prompt in seconds -- drop `--no-autoload` so the newest >>> snapshot is restored:

```
./out/build/relwithdebinfo/EmulatR.exe --firmware firmware/ds10_v7_3.exe --autosnapsh
```

Bounded headless cold boot -- determinism / step-correctness over a few minutes (covers decompression and early init):

```
EMULATR_NO_PUTTY=1 ./out/build/relwithdebinfo/EmulatR.exe \
  --firmware firmware/ds10_v7_3.exe --no-autoload --autosnapshot off \
  --max-cycles 2000000000 2> run_cold.log
```

See the Environment Variables reference for the full set of launch gates.

See Also.

[Connecting to the Console](#)  21

[Environment Variables](#)  55

[First Startup](#)  15

3.2 Loading Firmware

Pass the SRM image explicitly so the path resolves regardless of the ini default:

```
--firmware firmware/ds10_v7_3.exe
```

Relying on the ini fallback resolves the bare name `ds10_v7_3.exe` against the current directory, so an explicit `--firmware` is the reliable form. See Firmware Requirements for selection precedence and the optional SHA-256 integrity check.

See Also.

[Firmware Requirements](#)  14

[Reaching the SRM Prompt](#)  21

3.3 Running from the shell

EmulatR is a console application driven from a command line. The standard execution shell is `bash` -- the project's own launch and trace scripts are `bash`, and the examples here assume it (on Windows, a `bash` shell such as Git Bash / MSYS, where the disk appears as `/d/EmulatR`). Everything below is run from the `run` directory.

The run directory

The `run` directory is the folder that holds the built executable. CMake's post-build step lays it out so the program and its inputs sit together:

```
<run>/ Emulatr.exe the emulator
firmware/ SRM images (e.g. ds10_v7_3.exe)
tools/ deployed bash helpers (trace_halt.sh, ...)
EmulatrV4.ini settings (memorySize, paths, ...)
ds10_flash.rom persisted NVRAM / flash
snapshots/ saved machine states (.axpsnap)
```

Always launch from this directory (`cd <run>` first), because the default firmware path and the ini are resolved relative to the current directory. The bundled scripts in `tools/` compute the `run-dir` root from their own location (`$SCRIPT_DIR/..`) and `cd` there before running.

Basic invocation

```
cd /d/EmulatR/<run>
```

```
./Emulatr.exe --firmware firmware/dsl0_v7_3.exe
```

With no `--firmware`, the image falls back to `[Rom] firmwareImage` in the ini; with no `--mem`, memory falls back to `[System] memorySize` in the ini (1024 MB by default), and only then to the built-in 64 MiB default. The command line always wins over the ini.

Command-line switches

Switch	Meaning
<code>--firmware <path></code>	Firmware image to load (resolved against the run directory).
<code>--firmware-format auto raw srm</code>	Override image-format detection (default auto -- SRM if the <code>.exe</code> signature scan succeeds, else raw).
<code>--mem <bytes></code>	Guest memory size in bytes (e.g. 1073741824 for 1 GiB).
<code>--load-pa <addr> / --start-pa <addr></code>	Physical load address and execution start address (hex or decimal; start-pa low bit sets PAL mode).

Switch	Meaning
<code>--max-cycles <N></code>	Cap the run loop at N cycles (default unbounded).
<code>--pal-mode</code>	Force PAL mode at reset.
<code>--no-autoload</code>	Suppress autoload of the newest snapshot -- forces a genuine cold boot.
<code>--autosnapshot on off</code>	Toggle periodic <code>auto_*.axp</code> <code>snap</code> saves (off avoids the disk cost on a long cold boot).
<code>--snapshot-on-pc <pc[,pc...]></code>	Capture a one-shot snapshot when the CPU retires at the given PC(s).
<code>--dump-disasm <pa>:<count></code>	Disassemble count instructions at a PA after load, before the run starts.
<code>--trace <dec,machine></code>	Install the trace sink writing the decode and

Switch	Meaning
	machine listing logs.
--log-disable / --log-only / --log-verbose / --log-file	Tune the log subsystem (see Logging below).
--help	Print the usage text and exit.

Logging

The simplest log is the console transcript, captured with `tee` so it is both shown and saved:

```
./Emulatr.exe --firmware firmware/ds10_v7_3.exe 2>&1 | tee run_$(date +%Y%m%d_%H%M%S)
```

The log subsystem is filtered per channel. Show only certain subsystems, silence noisy ones, or send a channel to its own file:

```
./Emulatr.exe --firmware firmware/ds10_v7_3.exe --log-only ide,iic
./Emulatr.exe --firmware firmware/ds10_v7_3.exe --log-disable chipset
./Emulatr.exe --firmware firmware/ds10_v7_3.exe --log-file ide=logs/ide.log
```

Retire / instruction traces are written to the directory named by `EMULATR_TRACE_DIR`; `EMULATR_TRACE_WINDOW=1` arms a windowed sink so a cold boot does not emit a multi-gigabyte stream. A typical traced cold boot:

```
export EMULATR_TRACE_DIR=/d/EmulatR/traces; mkdir -p "$EMULATR_TRACE_DIR"
export EMULATR_TRACE_WINDOW=1
./Emulatr.exe --no-autoload --firmware firmware/ds10_v7_3.exe 2>&1 | tee cold_$(date -
```

The full list of `EMULATR_*` runtime variables (console port, flash path, idle warp, trace controls) is on the Environment Variables page.

Running as a forked / background process

Because the console is a RAW TCP server (reached by a terminal client over `EMULATR_CONSOLE_PORT`), the emulator does not need the controlling terminal and can run detached. Launch it in the background, capture its output to a file, and note the process id:

```
./Emulatr.exe --firmware firmware/ds10_v7_3.exe > run.log 2>&1 &
echo "emulatr pid = $!"
```

To survive the shell closing, use `nohup` and `disown`, then connect from a separate terminal client:

```
nohup ./Emulatr.exe --firmware firmware/ds10_v7_3.exe > run.log 2>&1 & disown  
# then, from anywhere: putty -raw 127.0.0.1 -P 10023 (or any RAW-TCP client)
```

Stop a detached instance cleanly by ending its console session or signalling the captured pid; avoid a hard kill during a flash write, which can corrupt `ds10_flash.rom`. To run two instances from one directory, give each its own flash and console port so they do not collide:

```
EMULATR_FLASH_ROM=flash_b.rom EMULATR_CONSOLE_PORT=10024 \  
./Emulatr.exe --firmware firmware/ds10_v7_3.exe > run_b.log 2>&1 &
```

See Also.

[Environment Variables \(EMULATR_*\)](#) 

[Executing Two Instances](#) 

[Configuring](#) 

3.4 Connecting to the Console

The emulator exposes a RAW TCP console (default `localhost:10023`). Unless `EMULATR_NO_PUTTY` is set, PuTTY is launched for you. To attach a client manually:

```
plink -raw localhost -P 10023
```

Full client coverage (PuTTY, plink, Tera Term, MobaXterm, netcat, telnet) is in the Console Connectivity chapter.

See Also.

[Loading Firmware](#) 

[TCP Console Overview](#) 

[Terminal Clients](#) 

3.5 Reaching the SRM Prompt

After launch the firmware proceeds through power-up to the console. The visible sequence is:

- *Banner and memory sizing (e.g., "1024 Meg of system memory").*
- *PCI / ISA probe; GCT / FRU initialization at 3f32000.*
- *Option-firmware scan of the CD / floppy devices.*
- *The Loadable Firmware Update (LFU) utility -- the UPD> prompt. From here, exit returns to the console, or update srm refreshes firmware.*

- *System / disk testing, then the interactive >>> prompt.*

A full annotated transcript is in Example - Performance. By default the console halts at >>> (auto_action = HALT).

See Also.

[Clean Shutdown](#)  22

[SRM Console](#)  30

[Firmware Update Utility \(LFU\)](#)  40

3.6 Clean Shutdown

There are two graceful paths and one hard kill.

1. Stop sentinel (flushes NVRAM via ~Machine)

```
touch /d/EmulatR/EmulatRAppUniV4/Emulatr/EMULATR_STOP
```

The run loop polls the sentinel even mid-stall, exits within a moment, and writes the flash cleanly. The resolved sentinel path is logged at startup (grep graceful-stop run.log).

2. CALL_PAL HALT at the prompt

Halting at >>> exits through ~Machine with a clean flush -- no sentinel touch needed.

3. Hard kill (instant, no flush)

```
taskkill //F //IM Emulatr.exe
taskkill //F //IM PuTTY.exe
```

See Also.

[Launching EmulatR](#)  16

[Kill the EmulatR \(appendix\)](#)  68

4 Operating EmulatR

Day-to-day tasks once EmulatR is built and runs: configuring the machine, attaching a console, driving the SRM console, managing storage and snapshots, updating firmware, and turning on logging and diagnostics. Each task has its own topic.

In this section

- *Configuring & Running -- the platform manifest and the ways to launch a run.*
- *Console Connectivity -- the TCP console and the terminal clients that attach to it.*
- *SRM Console -- the commands available at the prompt.*
- *Storage -- disks, CD-ROM, and the storage devices.*
- *Snapshots -- saving and restoring machine state.*
- *Firmware Update Utility (LFU) -- updating the SRM image in flash.*
- *Guest Boot -- booting an installed guest.*
- *Logging & Diagnostics -- the trace and diagnostic facilities.*

See Also.

[Configuring & Running](#)  23

[Console Connectivity](#)  28

[SRM Console](#)  30

[Logging & Diagnostics](#)  43

4.1 Configuring & Running

How the emulated machine is configured by its platform manifest, and the ways to launch a run -- a quiet cold boot, a traced boot, a long unattended run, or two instances side by side.

In this section

- *Configuring -- the per-model platform manifest and its key settings.*
- *Executing -- run scenarios beyond the basic launch: traced boots, unattended runs, performance harvesting.*
- *Executing Two Instances -- running two instances safely from one directory.*
- *Example - Performance -- an annotated boot transcript.*

See Also.

[Configuring](#)  24

[Executing](#) ²⁵

[Launching EmulatR](#) ¹⁶

4.1.1 Configuring

Configuration precedence (lowest to highest): compiled-in struct defaults < `EmulatrV4.ini` < command-line qualifiers. The committed ini drives a no-flag launch; a CLI qualifier overrides the ini for a single run. `IniLoader` populates `EmulatorSettings`, which is the single source of truth threaded to every consumer.

Settings

Field	ini Key	Default	Example	CLI / Env override
model	[System]/model	DS10	ds10	
cpuCount	[System]/cpuCount	1		
port	[SRMConsole]/port	10023		EMULATR_CONSOLE_PORT
puttyPath	[SRMConsole]/puttyPath	putty.exe	C:\Program Files\PuTTY\putty.exe	
firmwareImage	[ROM]/firmwareImage	ds10_v7_3.exe		--firmware
firmwareSha256	[ROM]/firmwareSha256	(empty -- integrity check skipped)		

Command-line qualifiers override the ini for a single run. Live today: --firmware (firmware image path) and --mem (memory size). Planned (locked): --console-port, --platform-config, --flash-path; the EMULATR_CONSOLE_PORT / EMULATR_FLASH_ROM / EMULATR_PLATFORM_CONFIG environment variables remain as documented aliases.

Debug-only environment variables (EMULATR_IDE_TRACE, EMULATR_IIC_TRACE, EMULATR_FLASH_TRACE, EMULATR_NO_PUTTY) are diagnostic toggles, not configuration: they have no ini key, no CLI qualifier, and no precedence interaction.

Bus configuration (platform manifest)

The bus configuration is loaded for the configured model. The platform file is selected from the model name as `lower(model) + "_platform.json"` (for example, DS10 gives `ds10_platform.json`; ES40 and ES45 give `es40_platform.json` and `es45_platform.json`). An explicit override wins, in order: the `EMULATR_PLATFORM_CONFIG` environment variable, then a `platformConfigPath` ini key, then the derived name. A missing platform file is a hard error -- there is no silent fallback.

See Also.

[Current Platform Support](#) ¹⁰

[Executing](#) ²⁵

[Environment Variables](#) ⁵⁵

4.1.2 Executing

Beyond the basic launch (see Launching EmulatR), a few run scenarios come up often. All are launched from the EmulatR root and write diagnostics to a redirected log.

Traced boot

Turn on the device trace channels and put the console on an alternate port (so it coexists with another instance). PuTTY follows `EMULATR_CONSOLE_PORT`.

```
EMULATR_NO_AUTOLOAD=1 EMULATR_CONSOLE_PORT=10024 \
  EMULATR_IDE_TRACE=1 EMULATR_IIC_TRACE=1 EMULATR_FLASH_TRACE=1 \
  ./out/build/release/EmulatR.exe \
  --firmware firmware/ds10_v7_3.exe --autosnapshot off 2> run_ide.log
```

Unattended (overnight) run

Use `nohup` so the run survives logoff; leave autosnapshot on and mirror the console to the log.

```
nohup env EMULATR_NO_AUTOLOAD=1 EMULATR_CONSOLE_SNAPSHOT=1 \
  EMULATR_CONSOLE_MIRROR=1 EMULATR_CONSOLE_PORT=10023 \
  ./out/build/relwithdebinfo/EmulatR.exe --firmware firmware/ds10_v7_3.exe \
  > run_overnight.log 2>&1 &
```

Harvest the slowest console phases

After a run with the console mirrored to the log, the slowest timestamped phases fall out of the log with:

```
grep -a '\[CON' boot.log \
  | sed -E 's/.*\[CON [^+]*\+ *([0-9.]+) ms\[ ?/\1\t/' \
  | sort -rn | head -25
```

See Also.

[Launching EmulatR](#) 

[Executing Two Instances](#) 

[Environment Variables](#) 

[Running from the shell](#) 

4.1.3 Executing Two Instances

Two instances launched from the same directory share `ds10_flash.rom` and the `snapshots/` directory, and concurrent NVRAM writes can corrupt the flash. Give each console its own port with `EMULATR_CONSOLE_PORT`, and if a run will write NVRAM (any `set` or `update` at the `>>>` prompt), isolate its flash first.

```
cp ds10_flash.rom ds10_flash_release.rom
# then prepend on the launch line: EMULATR_FLASH_ROM=ds10_flash_release.rom
```

With a separate `EMULATR_FLASH_ROM` per instance, NVRAM writes no longer collide. Confirm the second instance came up on its own port by checking its log for "Listening on TCP port 10024".

See Also.

[Executing](#)  25

[Snapshots](#)  39

[Clean Shutdown](#)  22

4.1.4 Example - Performance

A representative cold-boot transcript: power-up, the Loadable Firmware Update (LFU) utility, an `update srm`, and the interactive `>>>` prompt with a few `show / set` commands. Spacing is simplified for readability.

```
ASA EmulatR -- Alpha AXP (EV6 / 21264) Emulator
Alpha Emulator Console V4.0-0
(c) 2026 Timothy Peer / eNVy Systems, Inc.
```

```
*** keyboard not plugged in...
Flash ROM writes are disabled
1024 Meg of system memory
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
initializing GCT/FRU at 3f32000
```

```
Checking dqa0.0.0.105.0 for the option firmware files. . .
dqa0.0.0.105.0 has no media present or is disabled via the RUN/STOP switch
Checking dva0.0.0.0.0 for the option firmware files. . .
Option firmware files were not found on CD or floppy.
... or just hit <return> to proceed with a standard console update:
```

```
***** Loadable Firmware Update Utility *****
Function Description
Display Displays the system's configuration table.
Exit Done exit LFU (reset).
List Lists the device, revision, firmware name, and update revision.
Update Replaces current firmware with loadable data image.
Verify Compares loadable and hardware images.
```

```
UPD> update srm
Confirm update on:
srm [Y/(N)]y
WARNING: updates may take several minutes to complete for each device.
DO NOT ABORT!
srm Updating to 7.3-1... Verifying 7.3-1... PASSED.
UPD> exit
```

```
Initializing....
```

```
Testing the System
Testing the Disks (read only)
snapshot Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored

>>>show config
SRM Console: V7.3-2
PALcode: OpenVMS PALcode V1.98-83, Tru64 UNIX PALcode V1.92-73

Processors
CPU 0 Alpha EV6 pass 1 268 MHz
  SROM Revision: P<?>
  Bcache is disabled

Core Logic
Cchip DECchip 21272-CA Rev 2
Dchip DECchip 21272-DA Rev 1
Pchip 0 DECchip 21272-EA Rev 1
TIG Rev 7.31
Arbiter Rev 7.31 (0xff)

MEMORY
Array # Size Base Addr
  0 1024 MB 0000000000
Total Good Memory = 1024 MBytes

PCI Hose 00
  Bus 00 Slot 05/0: Cypress 82C693
  Bridge to Bus 1, ISA
  Bus 00 Slot 05/1: Cypress 82C693 IDE
  dqa0.0.0.105.0 EMULATR VIRTUAL CDROM

ISA (Slot/Device/Name/Type/Enabled/BaseAddr/IRQ/DMA)
  0 MOUSE Embedded Yes 60 12
  1 KBD Embedded Yes 60 1
  2 COM1 Embedded Yes 3f8 4
  3 COM2 Embedded Yes 2f8 3
  4 LPT1 Embedded Yes 3bc 7
  5 FLOPPY Embedded Yes 3f0 6 2

>>>show device
dqa0.0.0.105.0 DQA0 EMULATR VIRTUAL CDROM
dva0.0.0.0.0 DVA0
>>>show memory_test
memory_test full
>>>set memory_test none
>>>set full_powerup_diags off
>>>show memory_test
memory_test none
>>>
```

See Also.

[Reaching the SRM Prompt](#)  ²¹[SRM Console](#)  ³⁰[Current Development Status](#)  ¹¹

4.2 Console Connectivity

EmulatR serves its console over a RAW TCP socket. This subsection covers what the console is, how the port is chosen, and how to attach each common terminal client.

In this section

- *Console Support* -- what the console is and which clients work.
- *TCP Console Overview* -- the port, the auto-launch, and running multiple instances.
- *Terminal Clients* -- step-by-step connection for PuTTY, Plink, Tera Term, MobaXterm, and netcat.

See Also.

[Console Support](#)  ²⁸[TCP Console Overview](#)  ²⁹[Terminal Clients](#)  ²⁹

4.2.1 Console Support

EmulatR exposes its console as a RAW TCP stream (default `localhost:10023`) -- there is no Telnet option negotiation, so a raw-capable client is preferred. Any of the following can attach:

- *PuTTY (Raw)*
- *Plink (-raw)*
- *Tera Term (TCP/IP)*
- *MobaXterm (TCP)*
- *netcat (nc)* -- handy for scripting and automated testing
- *telnet* (may connect, but raw is preferred)

If EmulatR runs on another machine, replace `localhost` with that host's name or IP. For development and debugging the recommended client is Plink with verbose logging:

```
plink -v -raw localhost -P 10023
```

See TCP Console Overview for the port and auto-launch behavior, and Terminal Clients for the step-by-step connection in each program.

See Also.

[TCP Console Overview](#) [Terminal Clients](#) [Connecting to the Console](#) 

4.2.2 TCP Console Overview

EmulatR serves the console over a RAW TCP socket -- no Telnet option negotiation.
Defaults:

```
Host: localhost
Port: 10023
Protocol: RAW TCP
```

Override the port with `EMULATR_CONSOLE_PORT` (the auto-launched PuTTY follows the port).
Suppress the PuTTY auto-launch with `EMULATR_NO_PUTTY`; the listen socket still comes up for your own client. For a remote host, replace `localhost` with its hostname or IP.

Port `10023` is only the default -- the listener can bind any available TCP port. This is what makes running several instances at once practical: give each its own port (`10023`, `10024`, `10025`, and so on) so their consoles do not collide. Pick a free port above the privileged range (`1024` and up); confirm it is free before binding:

```
netstat -ano | grep <port>
```

When connecting, point the client at the same port the instance was started with.

See Also.

[Terminal Clients](#) [Executing Two Instances](#) [Configuring](#) 

4.2.3 PuTTY / Plink / TeraTerm / MobaXterm / Netcat

Connection steps for the common clients. In every case the target is `localhost` port `10023` (or a remote host and the port the instance was started with), using a RAW connection.

Plink

```
plink -raw localhost -P 10023 # local
plink -raw 192.168.1.50 -P 10023 # remote
plink -v -raw localhost -P 10023 -log emulator.log # verbose + session log
```

Useful arguments: `-raw` (raw TCP stream), `-P <port>` (TCP port), `-v` (verbose diagnostics), `-log <file>` (save session log).

PuTTY

Launch PuTTY, set Host Name `localhost` and Port `10023`, choose Connection type **Raw**, then press Open. For a remote instance use its IP in place of `localhost`.

Tera Term

New connection, select **TCP/IP**, Host `localhost`, TCP Port `10023`, then connect.

MobaXterm

Session, then a Network session: Remote host `localhost`, Port `10023`, Protocol TCP, then connect.

Netcat (Linux / macOS)

```
nc localhost 10023 # local
nc 192.168.1.50 10023 # remote
```

Useful for scripting and automated testing.

Telnet

Some telnet clients connect (`telnet localhost 10023`), but because EmulatR uses a raw socket and performs no Telnet option negotiation, a raw-capable client (PuTTY Raw, Plink - `raw`, Tera Term TCP/IP, MobaXterm TCP) is preferred.

Verifying the connection

After connecting you should see console output -- a banner, the `>>>` prompt, or an in-progress power-up sequence -- depending on the current emulator state.

Troubleshooting

Connection refused: verify EmulatR is running, the console server is enabled, and the port number is correct.

Connection timeout: verify firewall configuration and the host address, and that the listener is active.

No console output: verify the boot has progressed, console output is enabled, and you are on the right port.

See Also.

[TCP Console Overview](#) 

[Console Connection Issues](#) 

4.3 SRM Console

At the `>>>` prompt the console accepts `show` and `set` commands plus device and diagnostic commands; variable changes persist to the flash / NVRAM image. This subsection covers the console and its everyday commands.

In this section

- *SRM Overview -- what the console is and its reported versions.*
- *Common Commands -- the command set at a glance.*

- *show config -- the system configuration table.*
- *show device -- enumerated devices.*
- *show memory -- the memory array.*
- *boot -- bootstrap (planned; HALT today).*
- *set -- change and persist a console variable.*
- *SRM Environment Variables -- where the full variable list lives.*

See Also.

[SRM Overview](#) 

[Common Commands](#) 

[Example - Performance](#) 

4.3.1 SRM Overview

At >>> you interact with the console using `show` (display a variable), `set` (change a variable), and `device` / diagnostic commands. Settings persist to the flash / NVRAM image. The default power-up action is HALT, so the console waits at >>> for an operator command.

The reported console version is v7.3-2 (Feb 27 2007); PALcode is OpenVMS V1.98-83 / Tru64 UNIX V1.92-73.

See Also.

[Common Commands](#) 

[SRM Environment Variables](#) 

4.3.2 Common Commands

The commands used most often at the >>> prompt. Each `show` command has its own page in this subsection.

- *show <var> -- display the stored and active value of a console variable.*
- *set <var> <value> -- change a variable (persisted to NVRAM).*
- *show config -- system configuration table (CPU, core logic, memory, PCI / ISA).*
- *show device -- enumerated storage / I/O devices.*
- *show memory -- memory array layout.*
- *boot -- bootstrap the OS (planned on EmulatR; HALT is the implemented action).*
- *help / ? -- console help.*

See Also.[show config](#)^[32][set](#)^[33][SRM Environment Variables](#)^[33]**4.3.3 show config**

`show config` prints the full configuration table: the SRM and PALcode versions; the processor (Alpha EV6 pass 1, 268 MHz); core logic (Cchip 21272-CA, Dchip 21272-DA, Pchip 21272-EA, TIG / Arbiter Rev 7.31); the memory array; the PCI hose (the Cypress 82C693 bridge and IDE at bus 0, slot 5); and the ISA device list (MOUSE, KBD, COM1, COM2, LPT1, FLOPPY with their base addresses and IRQs). A full sample is in Example - Performance.

See Also.[show device](#)^[32][show memory](#)^[32][Example - Performance](#)^[26]

Clock Frequency and the MHz Reading

4.3.4 show device

`show device` lists enumerated devices. On the DS10 target:

```
dqa0.0.0.105.0 DQA0 EMULATR VIRTUAL CDROM
dva0.0.0.0.0 DVA0
```

`dqa0` is the virtual ATAPI CD-ROM on the CY82C693 IDE channel; `dva0` is the floppy device. The `dqa0` enumeration path is under active work -- see Storage and Current Development Status.

See Also.[show config](#)^[32][Storage](#)^[34][Current Development Status](#)^[11]**4.3.5 show memory**

The memory array is reported in the configuration table:

```
Array # Size Base Addr
  0 1024 MB 000000000
Total Good Memory = 1024 MBytes, Total Bad Pages = 0
```

The default system memory is 1024 MB (set by `[System] memorySize` in the ini); the size can be overridden at launch with `--mem`.

See Also.

[show config](#)³²
[Configuring](#)²⁴

4.3.6 boot

On real SRM, `boot` bootstraps the operating system using `bootdef_dev` and `boot_osflags`. On EmulatR, OS bootstrap is not yet implemented -- the console reaches `>>>` and halts. The related `auto_action` variable supports HALT (the default and only fully implemented action); BOOT and RESTART are planned.

See Also.

[Current Development Status](#)¹¹
[Guest Boot](#)⁴²

4.3.7 set

`set` changes a console variable and persists it to the flash / NVRAM image; `show` reads it back. Example session:

```
>>>set oem_string snapshot
>>>set memory_test none
>>>set full_powerup_diags off
>>>show memory_test
memory_test none
```

Setting `memory_test none` and `full_powerup_diags off` shortens the power-up self-test -- useful for faster iteration (see Slow Boot).

See Also.

[SRM Environment Variables](#)³³
[Slow Boot](#)⁶²

4.3.8 SRM Environment Variables

The complete, annotated list of SRM console variables (`auto_action`, the `boot_*` set, `com1_*` / `com2_*`, the `d_*` diagnostics group, the N1 to N16 NVRAM slots, `wwid0` to `wwid7`, and so on) is maintained in the appendix:

- -Appendix, DS10, "SRM Commands >>> Prompt".

These SRM variables are distinct from the emulator's own `EMULATR_*` launch environment variables, which are documented under the Reference section's Environment Variables page.

See Also.

[set](#)³³

[Environment Variables](#) ⁵⁵

4.4 Storage

Storage on the DS10 target is presented through the Cypress CY82C693 southbridge IDE / ATAPI channels. This subsection covers creating and attaching disk and optical images and the per-bus device models. SCSI and the legacy buses are noted where their status is still to be confirmed.

In this section

- *Create Guest Disk Storage -- provision a disk image (manifest or mkdisk).*
- *Virtual Disk Images -- the raw image format and how a disk attaches.*
- *Virtual CDROM -- the dqa0 optical device and ISO media.*
- *IDE Devices / ATAPI Devices (+ ISO-9660) -- the CY82C693 channels.*
- *SCSI Devices and the legacy buses (SDI / DSSI / ST506).*

See Also.

[Create Guest Disk Storage](#) ³⁴

[Virtual Disk Images](#) ³⁶

[IDE Devices](#) ³⁷

4.4.1 Create Guest Disk Storage

EmulatR provisions disk images two ways: automatically at boot, driven by the platform manifest (`create_if_missing`), and manually with the `tools/mkdisk.py` generator. Both share one image-creation codepath and one geometry catalog, so a disk type's size is defined in exactly one place. A manifest references a disk **type**, not a raw byte count -- the size is resolved from the catalog at load.

Geometry catalog (single source of truth)

The authoritative geometry source is `config/disk_types.json`, generated at build time from `dec_disk_media_types.tsv`. Never hand-maintain both: edit the TSV, regenerate the JSON. Each type names `total_lbn` and `block_bytes`; capacity is derived (`total_lbn * block_bytes`). A type is either a real DEC model (for example RZ28) or a sized blank target (for example blank-4G).

type	interface	total_lbn	block_bytes	capacity
blank-4G	ata	8388608	512	4 GiB
RZ26	scsi	2050860	512	1.05 GB
RZ28	scsi	4110480	512	2.1 GB
RZ29	scsi	8407200	512	4.2 GB

Automatic provisioning (create_if_missing)

When a manifest storage entry names a missing image and sets `create_if_missing`, EmulatR provisions a blank sparse image of the catalogued size on first boot. It never overwrites an existing image, and never provisions read-only media (an ISO is supplied, not created). Example manifest entry for the dqa0 ATA disk:

```
{ "channel": 0, "unit": 0, "type": "ata_disk",  
  "media": "dqa0.img", "disk_type": "blank-4G",  
  "create_if_missing": true }
```

On first cold boot the run log reports the provisioning and the attach:

```
Storage: created blank disk image 'dqa0.img' (4294967296 bytes)  
attached ATA disk 'dqa0.img' to IDE ch0 unit0
```

Manual creation (tools/mkdisk.py)

Use `mkdisk.py` to pre-make a typed image outside the boot path -- a specific DEC model, or a sized install target. It calls the same core as `create_if_missing`: looks up `total_lbn * block_bytes` from the catalog, writes a sparse zero-filled file, refuses to overwrite without `--force`, never creates for read-only media, and self-checks that the final size equals the catalog size.

```
python tools/mkdisk.py --type blank-4G --out dqa0.img  
python tools/mkdisk.py --type RZ28 --out dka0.img
```

The image written is BARE: exactly `total_lbn * block_bytes`, zero-filled, with no partition table and no DEC disklabel. The guest OS install writes those structures. For the install path (boot the installer, install onto a blank target, boot the installed disk) a bare image is what you want.

Cross-platform sparseness

Image creation uses a truncate-to-size, which is genuinely sparse on Linux ext4 (a 4 GiB image does not write 4 GiB physically). On Windows NTFS the file is lazily zero-filled and is not flagged sparse unless `FSCTL_SET_SPARSE` is issued; `mkdisk.py` sets it so behavior is identical on Windows and Linux. In practice the image costs nothing on disk until the guest writes to it.

See Also.

[Virtual Disk Images](#) ³⁶

[Guest Boot](#) ⁴²

4.4.2 Virtual Disk Images

An emulated fixed disk is backed by a plain raw, flat image file: the byte at offset $\text{LBA} \times \text{sector-size}$ is the first byte of that logical block. The ATA fixed disk (device `dqa0`) uses 512-byte sectors and is read-write, so an operating-system installer can write to it and the writes persist.

Images are created from the geometry catalog -- automatically at boot via `create_if_missing`, or manually with `tools/mkdisk.py` (see [Create Guest Disk Storage](#)). The file is bare: no partition table or DEC disklabel until the guest install writes them.

Attaching an image

A manifest storage entry names the channel, unit, type, and the `media` file. With `type = ata_disk` the image is opened as a 512-byte read-write block device on the CY82C693 IDE channel and appears to the firmware as `dqa0`. On launch the boot log confirms the attach:

```
Storage: attached ATA disk 'dqa0.img' to IDE ch0 unit0
```

Note: a blank image is not bootable on its own -- an operating system is installed onto it (booted from an ISO on the optical device). Enumeration of a bootable disk is in progress; see [Current Development Status](#).

See Also.

[Create Guest Disk Storage](#)^[34]

[Virtual CDROM](#)^[36]

[Current Development Status](#)^[11]

4.4.3 Virtual CDROM

`dqa0` ("EMULATR VIRTUAL CDROM") is the virtual optical device the firmware probes for option firmware during power-up. When no media is present it reports "no media present or is disabled via the RUN/STOP switch".

An ISO image is supplied (not created) through a manifest storage entry: an ATAPI / CD-ROM `type` with the `media` set to the ISO file. The optical device is read-only, so `create_if_missing` never applies to it. ISO contents are read through the ISO-9660 path (see ISO-9660 PFD).

[TODO: confirm the exact manifest type/key for attaching an ISO to `dqa0` against the current build.]

See Also.

[ATAPI Devices](#)^[37]

[ISO-9660 PFD \(CD001\)](#)^[37]

[Create Guest Disk Storage](#)^[34]

4.4.4 IDE Devices

The CY82C693 provides the IDE controller, exposed on PCI bus 0, slot 5, function 1 ("Cypress 82C693 IDE" in `show config`). The primary channel is the `dqa` device path. IDE register / port behavior (`0x1F0` to `0x1F7` and PCI config) and absent-slave probing are part of the current device-enumeration work.

See Also.

[ATAPI Devices](#) ³⁷

[Virtual Disk Images](#) ³⁶

[Current Development Status](#) ¹¹

4.4.5 ATAPI Devices

The CD-ROM is presented as an ATAPI device on the IDE channel -- `dqa0.0.0.105.0`, "EMULATR VIRTUAL CDROM". ATAPI / IDE scaffolding for the **CY82C693** is in active development; the `dqa0` enumeration gap (the firmware does not consistently see the device during the option-firmware scan) is a known open item.

Support notes

The master (CD) ATAPI handshake works end to end:

Alt-status reads 0x50 (DRDY | DSC) -- present and ready, not 0xFF, not 0x00.

The 0xAA scratch write to cyl-low reads back 0xAA -- the presence gate passes.

IDENTIFY DEVICE (0xEC) returns status 0x41 (DRDY | ERR), error 0x04 (ABRT), and cyl bytes 0x14 / 0xEB -- the ATAPI signature is correctly posted on abort.

IDENTIFY PACKET DEVICE (0xA1) runs, then two PACKET (0xA0) commands. A REQUEST SENSE CDB (03 00 00 00 12 00) returns sense key 0x02 (NOT READY) / ASC 0x3A (MEDIUM NOT PRESENT), then status 0x50, phase Complete.

See Also.

[Virtual CDROM](#) ³⁶

[ISO-9660 PFD \(CD001\)](#) ³⁷

[Current Development Status](#) ¹¹

4.4.6 ISO-9660 PFD (CD001)

An ISO-9660 image begins with a 16-block (32 KiB) system area, after which LBA 16 holds the Primary Volume Descriptor (PVD). The firmware's first coherence check is the standard identifier "CD001" at offset 1 of that descriptor. `VirtualIsoDevice` serves the PVD so a `READ(10)` of LBA 16 returns it. The doctest that locks this behavior:

```
TEST_CASE("VirtualIsoDevice: serves the ISO-9660 PVD at LBA 16 (CD001)") {
    // System area = 16 zero blocks; LBA 16 = PVD.
```

```

std::vector<uint8_t> img(17 * 2048, 0);
uint8_t* pvd = img.data() + 16 * 2048;
pvd[0] = 0x01; // descriptor type = PVD
std::memcpy(pvd + 1, "CD001", 5); // standard identifier
pvd[6] = 0x01; // version
// block size 2048 (both-endian) at offset 128
VirtualIsoDevice cd; REQUIRE(cd.loadMedia("pvd.iso"));
uint8_t buf[2048]; ScsiCommand c = read10(16, 1, buf);
cd.handleCommand(c);
CHECK(c.status == ScsiStatus::Good);
CHECK(std::memcmp(buf + 1, "CD001", 5) == 0);
}

```

See Also.

[ATAPI Devices](#) 

[Virtual CDROM](#) 

4.4.7 SCSI Devices

The SRM console exposes SCSI-related variables (`scsi_poll`, `scsi_reset`), but EmulatR does not currently model a SCSI host adapter. [TODO: confirm -- SCSI is not implemented; treat as planned.] The `VirtualScsiDevice` endpoint class exists in the device library for the eventual `dka` path.

For reference, the DEC RZ-series drives a DS10-class machine would address over SCSI (all 512-byte blocks):

Model	Capacity	Notes
RZ22	104 MB	SCSI drive
RZ23	209 MB	SCSI drive
RZ24	245 MB	SCSI drive
RZ25	426 MB	SCSI drive
RZ26	1.05 GB	in geometry catalog
RZ28	2.1 GB	in geometry catalog
RZ29	4.2 GB	in geometry catalog

See Also.

[Create Guest Disk Storage](#) 

[Current Development Status](#) 

4.4.8 Storage Interconnect and Fabric

On the DS10 target the storage path is short and fixed: the EV6 reaches the PCI hose through the Pchip, the Cypress CY82C693 sits on PCI bus 0 (slot 5) as the PCI-to-ISA bridge and IDE controller, and the storage endpoints (the ATAPI CD-ROM and any ATA disk) hang

off the CY82C693 IDE channel. There is no separate storage switch or expander -- the "fabric" is just the IDE channel and its master / slave units.

SCSI (the `dka` path) and the legacy DEC interconnects are reference material for other platforms; on the DS10 they are not part of the active fabric. See SCSI Devices and the legacy-bus page for those.

See Also.

[IDE Devices](#) 

[SCSI Devices](#) 

[SDI / DSSI / ST506 Devices](#) 

4.4.9 SDI Devices / DSSI Devices / ST506 Devices

These are legacy DEC / PC disk interconnects, included here for reference. None is present on the DS10, which uses the CY82C693 IDE / ATAPI channel exclusively; they are not modeled by EmulatR.

- *-SDI (Standard Disk Interconnect) -- the serial interface DEC used between an HSC / KDM controller and RA-series disks on large VAX systems.*
- *DSSI (Digital Storage Systems Interconnect) -- a multi-host bus used to share RF-series disks across clustered VAX / Alpha nodes.*
- *ST506 -- the early MFM / RLL hard-disk interface of the PC / workstation era, predating IDE.*

[TODO: confirm whether any of these are planned for a non-DS10 platform profile; otherwise this page stays reference-only.]

See Also.

[Storage Interconnect and Fabric](#) 

[SCSI Devices](#) 

4.5 Snapshots

EmulatR can capture and restore full machine state to `.axpsnap` files, so a long cold boot need only be paid once. Snapshot behavior is controlled by the `EMULATR_*` environment variables and the matching `--snapshot` / `--autosnapshot` CLI flags.

In this section

-Snapshot Operations -- automatic saves, creating, restoring, and pruning.

See Also.

[Snapshot Operations](#) 

[Launching EmulatR](#) ¹⁶

4.5.1 Automatic / Creating / Restoring / Pruning

Automatic snapshots

Periodic `auto_*.axpsnap` saves are on by default. Disable them for clean cold-boot / profiling runs:

```
--autosnapshot off # or EMULATR_AUTOSNAP=off
```

On startup the newest snapshot is auto-loaded unless you force a cold boot:

```
--no-autoload # or EMULATR_NO_AUTOLOAD=1
```

Disabling autosnapshot on mint cold-boot runs avoids the per-save disk cost.

Creating snapshots

Two capture mechanisms beyond the automatic saves:

- - *Console marker* -- arm with `EMULATR_CONSOLE_SNAPSHOT=1`; when a console input line matches the marker (default "set oem_string snapshot"), a `snapshots/predig_oemsnap_cyc<N>.axpsnap` is written. Change the marker with `EMULATR_SNAPSHOT_MARKER` (leading / trailing whitespace trimmed; internal whitespace is NOT normalized).
- *PC-triggered* -- `--snapshot-on-pc <pc>[...]`, optionally named with `--snapshot-name-tag <S>`.

Restoring snapshots

By default, startup restores the newest snapshot automatically (autoload-newest). To ignore snapshots and force a genuine cold boot, launch with `--no-autoload` (`EMULATR_NO_AUTOLOAD=1`). Snapshot files are resolved in the CWD-relative `snapshots/` directory, so launch from the EmulatR root for the canonical set.

Pruning snapshots

Pruning of old snapshots is off by default -- without the gate, `pruneOldSnapshots` is a no-op and nothing is evicted. Enable it with:

```
EMULATR_SNAPSHOT_PRUNE=1
```

See Also.

[Launching EmulatR](#) ¹⁶

[Executing Two Instances](#) ²⁵

[Environment Variables](#) ⁵⁵

4.6 Firmware Update Utility (LFU)

The Loadable Firmware Update (LFU) utility runs during power-up -- after the option-firmware scan finds nothing on CD or floppy, the firmware drops to the `UPD>` prompt. From

there you can list and update the firmware images in flash. `exit` leaves LFU (a reset) and continues to the console.

In this section

- *-LFU Commands -- the menu, List, Display, and Update.*

See Also.

[LFU Commands](#)  41

[Reaching the SRM Prompt](#)  21

[Example - Performance](#)  26

4.6.1 List / Display / Update

At the `UPD>` prompt the utility presents a function menu. `? or Help` scrolls it:

```
***** Loadable Firmware Update Utility *****
Function Description
Display Displays the system's configuration table.
Exit Done exit LFU (reset).
List Lists the device, revision, firmware name, and update revision.
Readme Lists important release information.
Update Replaces current firmware with loadable data image.
Verify Compares loadable and hardware images.
```

List

`list` shows each device with its current revision, firmware filename, and the revision available to update to:

```
UPD> list
Device Current Revision Filename Update Revision
nt Unknown nt_fw 5.71
srm 7.3-1 srm_fw 7.3-1
```

Display

`display` prints the system configuration table (the same PCI / ISA inventory shown by `show config` at the console). See `show config` and the Example - Performance transcript for the full layout.

Update

`update <device>` replaces the named device's firmware in flash. Updating `srm` refreshes the SRM console image; an unknown device name is rejected ("not supported under LFU").

```
UPD> update srm
Confirm update on:
srm [Y/(N)]y
WARNING: updates may take several minutes to complete for each device.
DO NOT ABORT!
srm Updating to 7.3-1... Verifying 7.3-1... PASSED.
```

When done, `exit` leaves LFU (a reset) and the firmware continues to the `>>>` prompt.

See Also.

[show config](#) 32

[Example - Performance](#) 26

[Firmware Requirements](#) 14

4.7 Guest Boot

From the SRM console prompt (`>>>`) you boot a device with `boot <device> [-fl root, flags]`. The device is a console device name; the optional boot flags select how the operating system bootstraps. Three SRM environment variables supply the defaults used when you type `boot` with no arguments.

Device names

`dqa0` is the IDE primary master (the ATA fixed disk); `dqa1` is the IDE primary slave (the ATAPI CD). `dka0` is the first SCSI disk; `ewa0` is the first Ethernet adapter (network boot). Booting the ATAPI CD (`dqa1`) is how an installer is started; booting `dqa0` boots the installed system.

SRM environment variables (the boot register settings)

Set with `set`, inspect with `show`. These persist in NVRAM across reboots.

variable	purpose	example
<code>bootdef_dev</code>	default boot device for a bare boot	<code>set bootdef_dev dqa0</code>
<code>boot_osflags</code>	default OS boot flags (root, flags for VMS)	<code>set boot_osflags "0,0"</code>
<code>auto_action</code>	action at power-on: HALT, BOOT, or RESTART	<code>set auto_action HALT</code>

Boot flags -- OpenVMS

OpenVMS flags take the form `root, flags`. `root` is the system root (`SYS0` .. `SYSn`, a single hex digit). `flags` is a hex bitmask; the load-bearing bit for interactive use is bit 0 (value 1), which requests a CONVERSATIONAL boot and stops at the `SYSBOOT>` prompt.

command	meaning
<code>boot dqa0 -fl 0,0</code>	normal boot from root <code>SYS0</code>
<code>boot dqa0 -fl 0,1</code>	conversational boot from <code>SYS0</code> -- stops at <code>SYSBOOT></code>
<code>boot dqa0 -fl n,1</code>	conversational boot from root <code>SYSn</code>

Other flag bits exist (debug, breakpoint, diagnostic-boot, memory-config display). The complete bitmask is platform-documented; consult the OpenVMS Alpha Upgrade and Installation manual. [TODO: confirm the full bitmask before documenting beyond bit 0.]

Conversational boot and SYSBOOT> (OpenVMS)

The bootstrap sequence: VMB.EXE (the primary bootstrap, on the boot disk) locates SYS\$SYSTEM:SYSBOOT.EXE (the secondary bootstrap), loads it, and starts it. With the conversational flag set, the boot stops after loading SYSBOOT.EXE and displays `SYSBOOT>`. There you use SYSGEN SET and SHOW to adjust system parameters (the default parameter file on Alpha is SYS\$SYSTEM:ALPHAVMSSYS.PAR), then CONTINUE to complete the boot.

```
>>> boot dqa0 -fl 0,1
... SYSBOOT>
SYSBOOT> SET WRITESYSPARAMS 0
SYSBOOT> CONTINUE
```

Boot flags -- Tru64 UNIX

Tru64 (OSF/1) also boots from the SRM console, but it has no SYSGEN conversational prompt; its analog is an interactive boot to the single-user shell, from which the kernel and startup can be controlled before going multi-user. The exact Tru64 boot-flag letters and the device / PCI-config dependency are still being pinned. [TODO: confirm Tru64 boot flags and the single-user gate.]

Install from CD (the EmulatR acceptance path)

The standard install flow uses the ATAPI CD to boot an installer that writes onto the blank ATA disk, after which the installed disk is bootable:

```
>>> boot dqa1 (boot the installer from the CD / ISO)
... installer writes the system onto dqa0
>>> boot dqa0 (boot the installed system)
```

Requirement: a complete PCI interface

Reaching `SYSBOOT>` (and the OS device configuration that follows) requires the EmulatR PCI interface to be complete -- config cycles, BAR assignment, and dynamic BAR-to-range rebind. The SRM console probe is enough to enumerate `dqa`, but VMB, SYSBOOT, and the early OS adapter initialization traverse PCI config space to reach the boot device. Until PCI is complete, a conversational boot cannot be expected to reach `SYSBOOT>`. See Current Development Status.

See Also.

[Create Guest Disk Storage](#)³⁴

[boot](#)³³

[Current Development Status](#)¹¹

4.8 Logging & Diagnostics

EmulatR's diagnostics are driven by runtime `EMULATR_*` trace gates and a built-in retire profiler. Most output goes to the process's standard error, so a run is captured by redirecting stderr to a log (`2> run.log`).

In this section

- *Diagnostics & Debugging* -- the trace gates, probe / break gates, and the compile-time diagnostics.
- *SRM Performance Logging* -- where profiling output is written.

See Also.

[Diagnostics & Debugging](#) ⁴⁴

[Environment Variables](#) ⁵⁵

[Executing](#) ²⁵

4.8.1 SRM Performance Logging

Performance-run output depends on which "performance run" you mean.

EmulatR's own profiler

The `RetireProfiler` is an always-on retire-PC histogram in the `PipelineDriver`. It writes `profile_YYYYMMDD-HHMMSS_<tag>.txt` to the same resolved trace directory as the other sinks -- `EMULATR_RETIRE_TRACE_DIR` (the build-tree default otherwise). It moves automatically when that directory is overridden.

External profiling tools

A Software Verify Performance Validator (PV) run is not written by EmulatR -- that is an external tool profiling `Emulatr.exe`. Its session and results go wherever PV saves them (its `saveSession` path, or PV's default session folder under your Documents). Nothing in the codebase controls that; it is a PV-side setting.

See Also.

[Diagnostics & Debugging](#) ⁴⁴

[Example - Performance](#) ²⁶

4.8.2 Diagnostics & Debugging

Diagnostics are turned on with runtime `EMULATR_*` environment variables, set in the launching shell before the process starts. The complete table lives in the Reference section's Environment Variables page; this page groups them by what they do.

Three gotchas to know first

- *Presence, not value.* Most gates test only that the variable is set, so `VAR=0` still **ENABLES** them. The few that parse a value (the autosnapshot / marker / path / numeric knobs) are noted in the reference.

- *Read once.* The trace and probe gates cache their result on first call, so they are effectively read at process start; exporting them mid-run has no effect.
- *Paths are CWD-relative.* Launch from the EmulatR root so relative output and snapshot paths resolve to the canonical set.

Device / subsystem traces

Presence-gated, read once, written to stderr: `EMULATR_IIC_TRACE` (PCF8584 IIC bus transactions), `EMULATR_FLASH_TRACE` (FlashRom command / erase / write activity), `EMULATR_PIC_TRACE` (the 8259 PIC pair -- IRQ assert / EOI). Redirect to per-trace logs, for example `2> iic.log`.

Chipset diagnostics and retire trace

The chipset CSR diagnostic stream is on by default; `EMULATR_CHIPSET_DIAG_OFF` silences it (note the inverted sense -- setting it turns diagnostics OFF). `EMULATR_TRACE_WINDOW` forces the retire-compact trace window, and `EMULATR_RETIRE_TRACE_DIR` sets the output directory for the retire / breakpoint / profiler trace files (see SRM Performance Logging).

Probe and break gates

For pinpoint debugging: `EMULATR_GCT_WATCH` (store-watch on the GCT / FRU config-tree region), `EMULATR_BREAK_ON_PA10` (conditional break on accesses to PA 0x10..0x17), `EMULATR_CHECKPOINTS` (named PC checkpoints), and `EMULATR_GATE` (a PC / condition gate controlling trace windowing).

Compile-time diagnostics (NOT runtime env vars)

Some `EMULATR_*` names are CMake options baked in at build time -- setting them in the shell does nothing. The important one is `EMULATR_MEMDIAG`: a per-retired-instruction `fprintf+fflush` that imposes roughly a 900x slowdown and must stay OFF for normal use. `EMULATR_DIAGNOSTIC_LOGGING` (centralized per-subsystem logging, non-Release builds) and the `EMULATR_LOG_*` `spdlog` level macros are also build-time.

For traced-boot command recipes, see Executing.

See Also.

[Environment Variables](#) 

[SRM Performance Logging](#) 

[Executing](#) 

5 Architecture & Internals

How EmulatR is built and how a cold boot actually runs. These pages are the operator lens; it presents enough detail to reason about the machine without reading the source. The register-level detail lives in the Appendices' Architecture Deep-Dives.

In this section

- *Overview -- the fidelity standard, the goals, the execution model, and the device fabric.*
- *Memory -- the GuestMemory physical store and the chipset arbiter.*
- *Addressing -- EV6 translation and the SPAM translation buffers.*
- *Firmware Decode & Decompression -- how the SRM image unpacks at cold boot.*
- *Execution Flight Plan -- the cold-boot sequence end to end.*

See Also.

[Overview](#)  46

[Memory](#)  50

[Execution Flight Plan](#)  52

5.1 Overview

EmulatR is a full-system, high-fidelity emulator of the DEC Alpha AXP platform -- specifically the 21264 (EV6) processor and the 21272 (Tsunami / Typhoon) core logic as integrated in AlphaServer DS10 / ES40 / ES45-class systems. It is implemented in modern C++ (C++20) with a deliberately framework-free architectural core (Qt is confined to the periphery -- console UI and host glue), built with CMake. Its purpose is not to approximate an Alpha; it is to be one, faithfully enough that unmodified DEC SRM console firmware, PALcode, and the operating systems that ran on the metal run on EmulatR without knowing the difference.

This page describes the architecture of the current implementation (V4) -- the execution model, the memory and translation system, the privileged boundary, and the device fabric -- and the standard every subsystem is measured against. The chapters that follow expand each area; the deep-dive appendices carry the register-level detail.

The fidelity standard and its authoritative sources

EmulatR is governed by one standard: it must behave like real Alpha hardware in every architecturally visible way, even when that complicates the implementation. Correctness is defined by primary sources -- the **Alpha Architecture Reference Manual** and the original **DEC SRM console and PALcode sources**. These define what correct behavior is.

The test is black-box and adversarial, conducted by the most demanding guests available: the SRM console and PALcode exercise the privileged architecture directly, and a booting operating system exercises the memory model, the trap machinery, and the device fabric under real workloads. A second emulator (AXPBox) is consulted only as a convenience cross-check for differential triage -- it is explicitly **not** an authoritative source, because another implementation can be wrong, whereas the architecture manual and the firmware sources cannot. If a guest can distinguish EmulatR from an AlphaServer through architecturally defined behavior, the emulator is wrong.

Primary goals and objectives

1. **-Architectural correctness first.** EmulatR models Alpha's architectural semantics, not a particular chip's timing. Correctness always outranks speed.
2. **-Deterministic, debuggable execution.** Execution is reproducible and inspectable. The run loop is staged and cycle-counted, and the hot path is deliberately shallow so a stack trace at any point stays short and legible.
3. **-Explicit, per-instruction semantics.** Every instruction resolves to one faithful grain leaf; behavior is never inferred from an operand-shape family. Architectural results and exceptions become visible only at the Writeback stage.
4. **-Separation of concerns.** Decode, integer, floating-point, memory, control, and privilege are isolated into well-defined functional domains -- the "Boxes" -- with one-way dependency flow. No domain reaches into another's internal state.
5. **-SMP-aware by construction.** Per-CPU state, CPU-local translation buffers, and load-locked / store-conditional reservation tracking are built in, so a multiprocessor build is an extension rather than a rewrite (see the scope note below).

The execution model

Execution is driven by `PipelineDriver` (`pipelineLib`) through six in-order stages: `IF` (instruction fetch), `DE` (decode / dispatch), `GR` (grain-ready: read the register file into the execution context), `EX` (execute), `MEM` (memory drain), and `WB` (writeback). Each instruction is carried through the six stages on a `PipelineSlot`; architectural state -- the register file and the PC -- is committed only at `WB`, the single well-defined commit point that makes exceptions precise and replay deterministic.

Dispatch is flat. At `DE` a decoded instruction is resolved through the grain system to a single grain leaf -- a pure function with the signature `BoxResult(InstructionGrain const&, ExecCtx const&)` -- that implements its semantics. Execution is partitioned into the functional Boxes, each a namespace of grain leaves rather than a class hierarchy: **IBox** (control flow -- branches, jumps), **EBox** (integer ALU), **FBox** (floating point, IEEE and VAX, over the `fpBox` backend), **MBox** (loads, stores, load-locked / store-conditional), **CBox** (branch prediction and target resolution), and **PaiBox** (the privileged boundary). Dependencies flow one way: every Box depends on `coreLib`; most also on `iprLib`; `MBox` additionally on `memoryLib`.

A deliberate consequence is that the hot path is shallow -- from the driver, through the stage, into a grain leaf, the call-stack depth is typically two and rarely exceeds four. That keeps traces legible (serving debuggability) and minimizes the per-instruction scaffolding that profiling has shown -- rather than the opcode bodies -- to dominate runtime (serving performance without compromising correctness). The grain leaves are the leaves of that call tree by construction.

Dispatch tables are generated, not hand-maintained. `GrainMasterV4.tsv` is the single source of truth for instruction semantics; the `genGrains.py` codegen reads it and emits the dense primary table (indexed by the 6-bit opcode) plus the sparse secondary tables for multi-function opcodes, including the personality-aware PALcode dispatch (OSF/1 and VMS). There is no runtime trace cache or JIT; the generated tables are the decode artifact.

Memory and address translation

Physical backing. `GuestMemory` (`memoryLib`) is the single physical-memory interface -- a passive byte-store that pages the guest's physical address space in on demand, in 64 KiB pages, committed only as the guest first touches them, so a multi-gigabyte machine costs only its working set. The pages are backed by host `VirtualAlloc` on Windows and by `mmap` on Linux / ARM hosts (with a portable fallback). The earlier `SafeMemory` layer has been removed. `GuestMemory` itself does no MMIO routing or fault generation: the `TsunamiChipset` system-bus arbiter decodes addresses, routes device traffic, and raises range faults before raw RAM is ever touched. A `LockMonitor` tracks per-CPU load-locked / store-conditional reservations at cache-line granularity.

Guest translation. Address translation is performed by `Ev6Translator` (`mmuLib`) as the EV6 architecture specifies: PAL-mode bypass, the canonical virtual-address window check, and superpage (kseg) mapping, with non-mapped accesses returning an ITB or DTB miss. The translation buffers are modeled by the **SPAM** cache -- Set-Partitioned Associative Memory -- as `SPAMShardManager` instances resident in `CpuState` (`itbMgr` and `dtbMgr`, each 16 shards by 8 ways = 128 entries). Invalidation is epoch-based and lazy (`TlbEpoch`): a TBIAP bumps a global counter and a TBIAP a per-process one, so a flush is $O(1)$ and stale entries simply fail the liveness test at lookup. The translation buffers are CPU-local by design.

Faults, exceptions, and the privileged boundary

There is no central fault dispatcher. A leaf, or the memory drainer that detects a fault, sets a `faultCode` on the instruction's `BoxResult` (for example a DTB miss or an unaligned access). The hardware exception entry points are compile-time constants in `Ev6EntryVectors.h` (the vectors at `0x100` through `0x780`, spaced `0x80` apart). At WB the driver inspects the fault: a halt sets the CPU's halted flag; otherwise, once `PAL_BASE` is established, control diverts to the computed entry vector. Privileged operations -- `CALL_PAL`, `HW_MTPR` / `HW_MFPR`, `HW_REI` -- are handled in `PalBox`, which applies IPR side-effects (`PAL_BASE` normalization, TLB invalidation, ASN context) inline. Internal processor registers are flat members of `CpuState`.

The machine and its device fabric

`Machine` (`systemLib`) is the top-level orchestrator: it owns the `CpuState`, the `GuestMemory`, and the `TsunamiChipset`, and provides firmware load, reset, and the step / run entry points. The chipset aggregates the three 21272 components -- the **Cchip** (system control, memory arbitration, interval timers, DRAM array configuration), the **Dchip** (data path), and the **Pchip** (PCI host bridge) -- together with the on-board devices. The PCI fabric is a standard bus / slot / function hierarchy (there is no separate "stratum" abstraction).

The southbridge is the **Cypress CY82C693** PCI-to-ISA bridge (`Cypress_CY82C693ISABridge`), which carries the ISA / SuperIO plane. The modeled devices include the CY82C693 IDE / ATAPI channel, the SuperIO controller (FDC / COM / LPT), the TOY / RTC, a 16550 UART for the serial console, the PCF8584 IIC controller, the flash / SRAM, and the dual 8259A interrupt controllers (`Pic8259Pair`). SCSI endpoints (`VirtualScsiDevice` and the ISO-CD variant) exist in the device library.

Lineage: four implementations, one set of objectives

The current emulator is the fourth implementation; the objectives above were fixed from the start, and each version reached for them by a different mechanism.

- *V1 dispatched each instruction through a virtual call to a per-instruction grain. It was correct and booted SRM, but the per-call overhead capped throughput.*
- *V2 chased performance by dispatching on instruction families (operand-shape signatures). It was fast, but families under-determined behavior -- instructions sharing a signature still needed per-instruction corner-case handling the model could not express.*
- *V3 tried to hybridize V1 and V2; the coupling was too tight and introduced silent-failure modes -- the worst outcome for an emulator whose value is fidelity.*
- *V4 keeps a flat dispatch path but resolves it to faithful per-instruction grain leaves via generated tables, with performance pursued as the sum of many marginal gains rather than one architectural bet. The dispatch mechanism evolved across all four; the objectives did not.*

Scope note: this page describes the V4 architecture and the standard it is held to. Some elements are infrastructure ahead of full activation -- multiprocessor execution and cross-CPU IPI, full PAL trap delivery, and SCSI / PCI device enumeration. For what is implemented and working in the current build versus what is still in progress, see [Current Development Status](#).

See Also.

[Memory](#) ⁵⁰

[Addressing](#) ⁵⁰

[Execution Flight Plan](#) ⁵²

[Current Development Status](#) ¹¹

5.2 Memory

`GuestMemory` (`memoryLib`) is the single physical-memory interface -- a passive byte-store with no opinion about devices or faults. It pages the guest's physical address space in on demand, in 64 KiB pages, committing a page only when the guest first touches it. So a machine configured for gigabytes costs only its working set on the host. Pages are backed by `VirtualAlloc` on Windows and `mmap` on Linux / ARM hosts (with a portable fallback). The earlier `SafeMemory` layer has been removed.

The arbiter owns routing, not memory

`GuestMemory` does no MMIO routing and raises no faults. The `TsunamiChipset` system-bus arbiter decodes every physical address first: it routes device traffic to the right model, services the chipset CSRs, and raises range faults for non-existent memory before raw RAM is ever touched. That separation is what keeps device / MMIO traffic from ever aliasing real RAM.

Array configuration

The amount of RAM the guest sees is set by the platform manifest and configured into the Cchip's Array Address Registers (AAR). The Tsunami Cchip supports up to four arrays of up to 1 GB each (4 GB maximum); accesses above the configured limit are Non-existent Memory (NXM). The configured size is what the firmware reports under `show memory` and in the configuration table. [TODO: confirm the shipped DS10 default array size against the current build.]

Reservations

A `LockMonitor` tracks load-locked / store-conditional reservations per CPU at cache-line (64-byte) granularity, for up to four CPUs -- the substrate for the EV6 `LDx_L` / `STx_C` primitives.

See Also.

[Overview](#) ⁴⁶

[Addressing](#) ⁵⁰

[show memory](#) ³²

5.3 Addressing

Virtual-to-physical translation is performed by `Ev6Translator` (`mmuLib`) exactly as the EV6 architecture specifies. In PAL mode the address bypasses translation (`VA = PA`). Otherwise the translator applies the canonical virtual-address window check and the superpage (`kseg`) mapping; an address that is neither a superpage nor a resident mapping returns an ITB or DTB miss for the PAL fill handler.

The translation buffers (SPAM)

The ITB and DTB are modeled by the **SPAM** cache -- Set-Partitioned Associative Memory -- as `SPAMShardManager` instances that live in `CpuState` (`itbMgr` and `dtbMgr`). Each is 16 shards by 8 ways = 128 entries: the manager hashes a lookup tag to pick a shard, and the shard does a short way-scan. The buffers are CPU-local by design -- each `CpuState` owns its own pair, and cross-CPU invalidation (in a future SMP build) would go through IPI machinery.

Invalidation is lazy (epoch-based)

Rather than walk and clear entries, invalidation bumps a counter (`TlbEpoch`): a full flush (TBIA) advances a global counter and a per-process flush (TBIAP) advances a process counter, so a flush is $O(1)$ and a stale entry simply fails its liveness test at the next lookup. Targeted single-entry flushes (TBIS / TBISI / TBISD) clear one slot. The page-table-entry format (`Ev6PteFormat`) carries the protection, granularity-hint, and fault bits the guest expects, and translation faults are delivered precisely at the Writeback stage.

See Also.

[Memory](#)  50

[Overview](#)  46

[Addressing & TLB \(appendix\)](#)  85

5.4 Firmware Decode & Decompression

The shipped SRM image is compressed. At every cold boot the firmware decompresses itself on the emulated CPU, using the original DEC SRM decompressor -- a Huffman / inflate engine (Mark Adler's inflate, version c10p1) wrapped by a DEC `decompress()` routine. EmulatR runs that code faithfully rather than substituting its own.

What the startup does

The guest assembly startup (`ev6_huf_decom.m64`) is more than a call to `decompress()`: it first sets up CPU and PAL state -- establishes `PAL_BASE`, sets the `I_CTL` SDE bits, flushes the ITB / DTB and the instruction cache, primes the shadow registers, and saves / restores the SROM parameters passed in registers R16-R21 -- and only then inflates the image. Any future "skip the inflate" optimization must preserve that exact end-state.

Cost

The inflate inner loop is about 4 million cycles per cold boot (the spin observed at PC `0x60111c`). That one-time cost is the main reason snapshots exist -- a saved post-decompression state lets you skip straight to the prompt on later runs.

The host oracle

To verify the decompressed bytes are correct without trusting EmulatR's own CPU (or a second emulator), the project builds the **same** DEC source as a native host program in

`tools/host_decompressor`. Its output matches EmulatR's decompressed image byte for byte. This is a stronger reference than AXPBox's `decompressed.rom`, which runs the guest decompressor on AXPBox's own emulated CPU. The oracle settled the "single-bit PAL corruption" question: the `hw_ret R2` instruction at PC `0xd954` is genuine firmware, not a decompression artifact.

A host-native decompression path (running the inflate off-CPU while preserving the PAL side-effects) is a possible future optimization; today the faithful on-CPU path is what runs.

See Also.

[Execution Flight Plan](#)  52

[Decompressor Internals \(appendix\)](#)  87

[Current Development Status](#)  11

5.5 Execution Flight Plan

A cold boot is a fixed sequence of stages from reset to the interactive prompt. Each instruction along the way is carried through the six in-order pipeline stages (IF, DE, GR, EX, MEM, WB) and commits at Writeback (see Overview); the stages below are the firmware-level milestones.

The stages

1. **Reset / SR0M hand-off.** The machine resets with the SR0M parameters in registers R16-R21 and enters the firmware startup.
2. **PAL and CPU setup.** The startup establishes `PAL_BASE`, sets the `I_CTL` SDE bits, flushes the ITB / DTB and instruction cache, and primes the shadow registers.
3. **Decompression.** The firmware inflates its own compressed image (about 4M cycles; see Firmware Decode & Decompression).
4. **Relocation.** A memory-copy moves the decompressed firmware to its run location and jumps to the SRM console entry.
5. **SRM power-up.** Banner and memory sizing; the PCI / ISA probe; GCT / FRU initialization; the option-firmware scan of the CD and floppy; the Loadable Firmware Update utility (`UPD>`); system and disk testing; and finally the interactive `>>>` prompt. (That visible sequence is detailed in Reaching the SRM Prompt.)

Where the flight currently ends

The cold boot reaches the `>>>` prompt and is fully interactive. The remaining runtime work downstream of decompression centers on a clock-interrupt return path (the `hw_ret R2` sequence) and, for booting a guest, the front-panel halt gate and PCI device enumeration. Those are tracked in Current Development Status.

See Also.

[Overview](#)  46

[Firmware Decode & Decompression](#)  51

[Reaching the SRM Prompt](#)  21

[Execution / Boot Internals \(appendix\)](#)  89

5.6 Clock Frequency MHz

The SRM banner and `show config` report a CPU speed -- on the DS10 profile, 268 MHz. That number is not a hardcoded banner string and it is not read from a stored constant; the firmware derives it by measuring the CPU cycle counter against the chipset interval-timer interrupts. This page explains how that figure is produced and how to change it.

The profile clock

EmulatR has one build-time constant for the modeled CPU frequency,

`EMULATR_PROFILE_ALPHA_CLOCK_HZ`, defined in `chipsetLib/Tsunami21272_CsrSpec.h`.

The current DS10 value is:

```
EMULATR_PROFILE_ALPHA_CLOCK_HZ = 268435456 (2^28 Hz = 268.435456 MHz)
```

Everything downstream -- the interval-timer cadence and the frequency the firmware reports -- follows from this one number.

The interval timer cadence

The Cchip interval timer is designed to fire at the architectural nominal of 1024 Hz. The cycle target per tick is the profile clock divided by 1024, rounded to the nearest power of two so the fire test is a single mask compare:

```
kCchipIntervalTimerCycles = kProfileAlphaClockHz / 1024
kCchipTimerBit = roundLog2Nearest(kCchipIntervalTimerCycles)
fire when: (cycleCount & kCchipTimerMask) == 0 && cycleCount != 0
```

At 2^{28} Hz this is exact: $2^{28} / 1024 = 2^{18}$, already a power of two, so the rounding introduces zero error and the timer fires every 2^{18} cycles -- a clean 1024 Hz.

How the firmware turns that into MHz

The SRM measures its own clock: it reads the CPU cycle counter (PCC / RPCC) and counts how many cycles elapse across a known number of 1024 Hz interval-timer interrupts, then multiplies cycles-per-tick by the tick rate.

```
measured clock = cycles-per-tick x tick-rate
                = 2^18 cycles x 1024 Hz
                = 2^28 Hz = 268.435456 MHz -> "268 MHz"
```

So the reported MHz is the profile clock, recovered by measurement -- the calibration path comes back to exactly the value that drove the timer in the first place.

Why not the HWRPB constant?

The Hardware Restart Parameter Block does carry a cycle-counter frequency field (`cycle_count_freq`) and an interval-clock field (`intrclock_freq`, interrupts per second times 4096), built by `HwrpbBuilder`. But the `cycle_count_freq` spec value is 500 MHz, while the banner shows 268 -- so on the DS10 the firmware is clearly using its own measurement rather than the stored constant. (Firmware that instead trusted the HWRPB field would report 500.) The interval-clock field is set consistently with the 1024 Hz cadence.

Changing the reported speed

Override the profile clock at configure time, for example for an ES40 profile:

```
cmake -DEMULATR_PROFILE_ALPHA_CLOCK_HZ=600000000 (ES40, ~600 MHz)
```

One caveat follows directly from the calibration mechanism: a clock that is not a power-of-two multiple of 1024 makes `kCchipIntervalTimerCycles` round to the nearest power of two, so the tick rate drifts slightly from 1024 Hz and the firmware's measured MHz drifts with it (for ES40 600 MHz the tick lands at bit 19, about 1144 Hz, within ~12 percent of nominal -- harmless to a rate-agnostic OS but visible if a guest reads the calibrated speed). Powers of two like 2^{28} calibrate exactly; other values trade a small reported-speed skew for the chosen cadence.

See Also.

[Execution / Boot Internals](#)  ⁸⁹

[Memory Map & IPRs](#)  ⁸³

[show config](#)  ³²

6 Reference

Look-up material: the full environment-variable table, a glossary of Alpha and EmulatR terms, the build and runtime dependencies, and the project's attributions.

In this section

- *Environment Variables* -- every `EMULATR_*` runtime gate and the compile-time flags.
- *Glossary* -- terms and acronyms.
- *Dependencies* -- the frameworks, libraries, and tools EmulatR relies on.
- *Attributions & Credits* -- sources, heritage, and licensing acknowledgements.

See Also.

[Environment Variables](#) 55

[Glossary](#) 57

[Dependencies](#) 58

[Attributions & Credits](#) 59

6.1 Environment Variables (EMULATR_*)

The runtime environment variables EmulatR reads at launch, plus the compile-time flags that look like env vars but are CMake options. All values are read with `std::getenv` -- set them in the launching shell before starting the process (inline bash assignment is the normal form).

Three gotchas

- **Presence, not value.** Most gates test only that the variable is set, so `VAR=0` still ENABLES them. Exceptions that parse a value are noted below.
- **Read once.** The trace / diagnostic gates cache their result on first call; exporting them mid-run has no effect.
- **Paths are CWD-relative.** The stop sentinel and the snapshots directory resolve against the launch directory; launch from the EmulatR root.

Snapshot / autoload

Variable	Default	Effect
EMULATR_CONSOLE_SNAPSHOT	off	Arms the console-marker snapshot watch; a matching input line writes a .axpsnap.
EMULATR_SNAPSHOT_MARKER	set oem_string snapshot	The exact line that triggers a console snapshot (internal whitespace NOT normalized).

Variable	Default	Effect
EMULATR_AUTOSNAP	on	=off disables periodic auto saves. Mirrors --autosnapshot off.
EMULATR_NO_AUTOLOAD	unset	Skip autoload-newest; force a genuine cold boot. Mirrors --no-autoload.
EMULATR_SNAPSHOT_PRUNE	unset	Enable pruning of old snapshots (off by default).

Process / platform

Variable	Default	Effect
EMULATR_STOP_FILE	EMULATR_STOP	Path of the graceful-stop sentinel; touch it to flush flash NVRAM and exit cleanly.
EMULATR_NO_PUTTY	unset	Do not auto-launch PuTTY; the listen socket still comes up.
EMULATR_CONSOLE_PORT	10023	TCP console port (PuTTY auto-launch follows it). Give each instance its own port.
EMULATR_FLASH_ROM	ds10_flash.rom	Backing file for the flash / NVRAM image. Isolate it per instance to avoid corruption.
EMULATR_PLATFORM_CONFIG	derived DS10	Override the platform manifest.

Logging / diagnostics and device traces

Variable	Effect
EMULATR_CHIPSET_DIAG_OFF	Silences the chipset CSR diagnostics (inverted sense: setting it turns them OFF).
EMULATR_TRACE_WINDOW	Forces the retire-compact trace window.
EMULATR_RETIRE_TRACE_DIR	Output directory for retire / breakpoint / profiler trace files.
EMULATR_IIC_TRACE	Trace PCF8584 IIC bus transactions.
EMULATR_FLASH_TRACE	Trace FlashRom command / erase / write activity.
EMULATR_PIC_TRACE	Trace the 8259 PIC pair (IRQ assert / EOI).

Probe / experimental

EMULATR_IDLEWARP fast-forwards the idle loop to the next timer tick (clean).

EMULATR_GCT_WATCH, EMULATR_BREAK_ON_PA10, EMULATR_CHECKPOINTS, and EMULATR_GATE

are pinpoint debug gates. Note: `EMULATR_RSCCWARP` is QUARANTINED (caused boot corruption) and `EMULATR_TICKWARP` is a deprecated no-op.

Compile-time flags (NOT runtime env vars)

These are CMake options baked in at build time -- setting them in the shell does nothing:

`EMULATR_MEMDIAG` (per-instruction logging, ~900x slowdown, keep OFF),
`EMULATR_DIAGNOSTIC_LOGGING` (non-Release per-subsystem logging), and the
`EMULATR_LOG_*` spdlog level macros.

CLI flags that mirror env vars

`--autosnapshot off = EMULATR_AUTOSNAP=off; --no-autoload = EMULATR_NO_AUTOLOAD=1. --max-cycles, --snapshot-on-pc, and --snapshot-name-tag` have no env equivalent.

See Also.

[Diagnostics & Debugging](#) ⁴⁴

[Executing](#) ²⁵

[Configuring](#) ²⁴

6.2 Glossary

Terms and acronyms used throughout this guide.

Alpha AXP -- Digital's 64-bit RISC architecture. EmulatR models the 21264 (EV6) generation.
21264 / EV6 (EV67) -- the Alpha CPU implementation EmulatR emulates; EV67 is the later 21264A.

21272 / Tsunami / Typhoon -- the core-logic chipset: Cchip (control), Dchip (data), Pchip (PCI).

Cchip / Dchip / Pchip -- the three 21272 components: system control and memory arbitration, the data path, and the PCI host bridge.

CY82C693 -- the Cypress PCI-to-ISA southbridge (also the IDE / ATAPI controller and SuperIO).

SRM -- the Alpha console firmware (the >>> prompt) that EmulatR runs unmodified.

PALcode -- Privileged Architecture Library code: the low-level routines (OpenVMS and Tru64 personalities) that implement privileged operations.

SROM -- the serial ROM whose parameters seed the CPU at reset (passed in registers R16-R21).

LFU / UPD> -- the Loadable Firmware Update utility and its prompt, used to update firmware in flash.

NVRAM -- the non-volatile store (backed by the flash image) holding SRM environment variables.

IPR -- Internal Processor Register, accessed in PAL mode via `HW_MTPR` / `HW_MFPR`.

ITB / DTB -- the instruction and data Translation Buffers (the EV6 TLBs).

TLB -- Translation Lookaside Buffer: the cache of virtual-to-physical mappings.

SPAM -- Set-Partitioned Associative Memory: EmulatR's sharded, set-associative TLB model.

kseg -- the EV6 superpage region where a virtual address maps directly to physical (no page table).

ASN -- Address Space Number, the tag distinguishing per-process translations.

Box -- a functional execution domain in EmulatR: IBox (control flow), EBox (integer), FBox (floating point), MBox (memory), CBox (cache / predict), PalBox (privileged).

grain -- a single per-opcode leaf function that implements one instruction's semantics.

GCT / FRU -- the firmware's Configuration Tree and Field-Replaceable-Unit data, built during power-up.

HWRPB -- the Hardware Restart Parameter Block, the firmware-to-OS handoff structure.

VMB / SYSBOOT -- the OpenVMS primary and secondary bootstraps; a conversational boot stops at `SYSBOOT>`.

dqa / dka / dva / ewa -- SRM device names: IDE / ATAPI, SCSI, floppy, and Ethernet respectively.

ATAPI -- the packet command set for optical / removable devices on an IDE channel.

RZ-series -- DEC SCSI disk models (RZ26, RZ28, RZ29, ...).

.axpsnap -- an EmulatR machine-state snapshot file.

NXM -- Non-eXistent Memory: a physical access above the configured RAM limit.

RPCC -- the Alpha cycle-counter instruction (Read Processor Cycle Counter).

See Also.

[Overview](#) 

[What is EmulatR](#) 

6.3 Dependencies

What EmulatR is built with and what it relies on at build and run time. Exact versions are to be confirmed against the current project files.

Component	Role	Notes
C++20 / MSVC	implementation language and compiler	[TODO: confirm minimum toolset]
CMake	build system	[TODO: confirm minimum version]
Visual Studio 2022	host toolchain / IDE	Windows host
Qt 6	console / UI periphery only	the emulator core is deliberately Qt-free
spdlog	logging infrastructure	level macros and channels
Berkeley SoftFloat	IEEE / VAX floating-point backend	selectable vs a native backend
Python 3	tooling (mkdisk.py, codegen)	build / utility scripts

Component	Role	Notes
PuTTY / plink	terminal client for the console	any RAW-TCP client also works
Ghidra	firmware reverse-engineering	optional, developer-side

EmulatR also depends on reference inputs it does not reimplement: the **Alpha Architecture Reference Manual** and the original **DEC SRM console / PALcode sources** (the authoritative specification and the firmware image it loads), and the `AXPBox` emulator used as a non-authoritative differential cross-check.

See Also.

[Required Tools & Frameworks](#)  13

[Attributions & Credits](#)  59

6.4 Attributions & Credits

EmulatR stands on a great deal of prior work. This page records the principal sources and acknowledgments.

Architecture and firmware heritage

Digital Equipment Corporation (DEC) / Compaq / HP -- the Alpha AXP architecture, the 21264 / 21272 hardware, the SRM console, and the PALcode. The Alpha Architecture Reference Manual and the DEC firmware sources are the authoritative references EmulatR is measured against.

The MILO and apisrm source trees -- reference firmware / PALcode sources consulted during development.

Libraries and algorithms

Mark Adler -- the inflate decompression engine (version c10p1) used by the DEC SRM decompressor and mirrored in the host oracle.

Berkeley SoftFloat (John Hauser) -- the IEEE / VAX floating-point reference implementation behind the FP backend.

spdlog -- the logging library.

Qt -- the UI / console framework at the periphery.

Tools

Ghidra -- firmware reverse-engineering. PuTTY / plink (Simon Tatham) -- the console terminal client. AXPBox -- consulted as a non-authoritative differential cross-check.

Performance Profiling - Performance Validator

<https://www.softwareverify.com/product/performance-validator/>

Documentation Tooling - Help & Manual <https://www.helpandmanual.com/>

EmulatR

System Architect - Timothy Peer / eNVy Systems, Inc.

Claude (AI code assist)

[TODO: confirm and list the exact license for each third-party component (SoftFloat, spdlog, Qt, inflate, Ghidra, PuTTY) and the provenance / licensing terms for the DEC firmware images and reference sources.]

See Also.

[Dependencies](#)  58

[Firmware Decode & Decompression](#)  51

7 Troubleshooting

What to check when EmulatR will not start, hangs, boots slowly, is missing a device, or refuses a boot. Several of these are expected behaviors of the current build rather than failures -- the pages say which.

In this section

EmulatR will not start -- launch-time failures.

Firmware Hangs, Slow Boot, and Missing Devices -- long pauses and absent devices.

Console Connection Issues -- can't attach a terminal.

Halt Button is IN, BOOT NOT POSSIBLE -- the halt-gate symptom.

Example (Console) execution -- a console debugging session.

evaluate Address -- the examine / deposit console commands.

See Also.

[EmulatR will not start](#)⁶¹

[Halt Button is IN, BOOT NOT POSSIBLE](#)⁶³

[Current Development Status](#)¹¹

7.1 EmulatR will not start

Console port already in use. Check with `netstat -ano | grep 10023` and free it, or set `EMULATR_CONSOLE_PORT`.

Firmware not found. The ini default is the bare name and resolves against the CWD -- pass -- firmware `./firmware/ds10_v7_3.exe` or launch from the Emulatr root.

Missing platform manifest. A missing `<model>_platform.json` is a hard error with no fallback.

*Two instances from one directory share `ds10_flash.rom` and `snapshots/`; concurrent NVRAM writes can corrupt the flash. Isolate with `EMULATR_FLASH_ROM=<copy>` (see *Executing Two Instances*).*

See Also.

[Firmware Requirements](#)¹⁴

[Executing Two Instances](#)²⁵

[Configuring](#)²⁴

7.2 Firmware Hangs / Slow Boot / Missing Devices

Firmware hangs (usually timed loops, not crashes)

Some long pauses are time-based firmware loops rather than emulator hangs:

- *-The option-firmware scan -- after "Checking dva0.0.0.0 for the option firmware files. . ." the firmware enters a krn\$_sleep-based wait (the 0x7bef0 software-tick loop). It is a timed sleep, not a device probe; a profiled gap of roughly 36 minutes has been observed here (open item).*
- *dqa0 "no media present" is followed by a stall (suspected timer loop).*
- *GCT/FRU initialization can be significantly delayed.*

Slow boot (shorten it)

Boot time is dominated by power-up diagnostics and several bounded timeout loops. To speed iteration, turn the heavy diagnostics off at the prompt:

```
>>> set memory_test none
>>> set full_powerup_diags off
```

Bounded device-wait loops (approximate wall-clock): dq_wait_on_busy ~5000 x 100us; dq_wait_for_drq ~5000 x 100-200us; dq_long_wait_on_busy ~310000 x 100us (about 31 s). The EMULATR_IDLEWARP gate fast-forwards idle waits without altering committed time.

Missing devices

If a device is absent from `show device`, check the IDE / ATAPI enumeration path. The dqa CD enumeration gap is a known open item: the firmware does not always observe the device during the option-firmware scan. Tracing 0x1F0..0x1F7 plus the PCI config of bus 0 / dev 5 / func 1 distinguishes a port-never-initialized case from a probe-protocol mismatch.

See Also.

[set](#)³³

[ATAPI Devices](#)³⁷

[Current Development Status](#)¹¹

7.3 Console Connection Issues

For refused, timed-out, or silent console connections, the full client coverage is in the Terminal Clients page. In brief, check three things:

The listener is up -- the log shows "Listening on TCP port ...".

The port matches your client (the default is 10023; an instance may have been started on another with EMULATR_CONSOLE_PORT).

The client is in RAW mode -- the socket is raw, with no Telnet option negotiation.

See Also.[Terminal Clients](#)²⁹[TCP Console Overview](#)²⁹

7.4 Halt Button is IN, BOOT NOT POSSIBLE

Symptom

At power-up the console prints "Halt Button is IN, AUTO_ACTION ignored", and at the >>> prompt any boot attempt (for example `b dqa1`) is refused with "Halt Button is IN, BOOT NOT POSSIBLE".

What it means

The firmware reads the front-panel RUN/STOP (halt) switch and finds it asserted (IN). A halted machine refuses to boot and ignores `auto_action --` on real hardware you would release the halt switch to allow a boot.

In EmulatR

This halt gate is under active investigation, and it is the current reason a boot at >>> does not yet proceed. The exact source the firmware tests for the halt condition is being pinned with an armed hardware trace before any fix is written. The `e pmem:80130000FF8` read below returning `7FFFFFFFFFFFFFFF` is part of that diagnostic (it arms the trace). A representative session:

```
AlphaServer DS10 268 MHz Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored
>>> e pmem:80130000FF8
pmem: 80130000FF8 7FFFFFFFFFFFFFFF
>>> b dqa1
Halt Button is IN, BOOT NOT POSSIBLE
>>>
```

Until the halt source is modeled, this message on a boot attempt is expected. Progress is tracked in Current Development Status.

See Also.[Current Development Status](#)¹¹[Guest Boot](#)⁴²[boot](#)³³

7.5 Example (Console) execution

A console debugging session on a current build (1024 MB, DS10 268 MHz). Note the cosmetic "ISA table corrupt" message at power-up (the firmware reinitializes the ISA table

to defaults), the "system serial number not set" advisory, and the use of `e` (examine) and `d` (deposit) to inspect physical memory.

```
1024 Meg of system memory
probing hose 0, PCI
probing PCI-to-ISA bridge, bus 1
ERROR: ISA table corrupt!
Initializing table to defaults
type >>>init to use these changes
bus 0, slot 5 -- dqa -- Cypress 82C693 IDE
*** system serial number not set. use set sys_serial_num command.
initializing GCT/FRU at 3ff32000
Checking dqal.1.0.105.0 for the option firmware files. . .
...
AlphaServer DS10 268 MHz Console V7.3-2, Feb 27 2007 13:25:53
Halt Button is IN, AUTO_ACTION ignored
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
>>> e pmem:801300003C0
pmem: 801300003C0 000000000000000000
>>> d pmem:80130000040 1
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
```

See [evaluate Address](#) for the command syntax, and [Halt Button is IN, BOOT NOT POSSIBLE](#) for the halt-gate context.

See Also.

[evaluate Address](#) ⁶⁴

[Halt Button is IN, BOOT NOT POSSIBLE](#) ⁶³

[Example - Performance](#) ²⁶

7.6 >>> Evaluate Address

At the `>>>` prompt the SRM console can read and write memory directly -- useful for inspecting device CSRs and firmware state during debugging.

Examine (e)

`e <space>:<address>` reads a location and prints its value. The `pmem:` space addresses physical memory (including device / chipset MMIO):

```
>>> e pmem:80130000040
pmem: 80130000040 000000000000000000
```

Deposit (d)

`d <space>:<address> <value>` writes a value to a location:

```
>>> d pmem:80130000040 1
```

These are the commands behind the halt-gate diagnostics (for example the arming read of `pmem:80130000FF8`). Be precise with the address and space prefix; an address that is not in an executable region is rejected ("not executable"), and a malformed line produces a syntax error.

See Also.

[Example \(Console\) execution](#) 

[Halt Button is IN, BOOT NOT POSSIBLE](#) 

8 Appendices - Developer Reference

The deep developer lens: per-platform reference material and register-level engineering notes that sit beneath the operator-facing chapters.

In this section

8.1 Platform Appendices -- per-model notes (DS10 in depth; DS20 / ES40 / ES45).

8.2 Subsystem Internals -- floating point, the grain codegen, CMake guards.

8.3 Architecture Deep-Dives -- the register-level companions to Architecture & Internals.

8.4 Tools -- developer tooling (Ghidra).

See Also.

[Architecture Deep-Dives](#)⁷⁹

[Architecture Overview](#)⁴⁶

[Ghidra](#)⁹¹

9 Platform Appendices

Per-model reference material. The DS10 is the only platform EmulatR currently models and drives end to end, so it carries a full appendix with detailed sub-topics; the larger Tsunami-family machines (DS20, ES40, ES45) are scaffolded but not yet validated and are documented here as planned work.

In this section

Appendix - DS10 -- the modeled platform: CPU, chipset, device manifest, and detail sub-topics.

Env Vars Operation Ref / Kill the EmulatR / Known Limitations -- DS10 operational reference.

SRM Boot / SRM Commands Prompt / SuperIO Controller -- DS10 firmware and SuperIO analysis.

Task Matrix / TimeOut Loops -- engineering status and the bounded firmware wait loops.

Appendix DS20 / ES40 / ES45 -- planned platforms (not yet validated).

See Also.

[Appendix - DS10](#) 

[Current Platform Support](#) 

9.1 Appendix - DS10

Reference material specific to the DS10 platform -- the only model EmulatR currently models and drives end to end. This page summarizes the hardware EmulatR presents and the device manifest that defines it; the sub-topics carry the environment-variable reference, the SRM and SuperIO analysis, and the engineering status.

Hardware summary

The AlphaServer DS10 is a single-processor Tsunami-family workstation. EmulatR presents a single 21264 (EV6) CPU, the 21272 (Tsunami) core-logic chipset, and a Cypress CY82C693 combination ISA bridge / IDE controller / SuperIO. The default configuration reports 1024 MB of memory and a 268 MHz cycle clock. The platform is defined declaratively by `ds10_platform.json`; a missing manifest is a hard startup error with no fallback.

PCI device manifest

All devices sit on hose 0, bus 0. From `ds10_platform.json`:

Slot.Func	Device	Vendor:Device	Class	Role
5.0	Cypress ISA bridge	0x1080:0xc693	0x060100	ISA bridge + embedded

Slot.Func	Device	Vendor:Device	Class	Role
				SuperIO (COM/LPT/FDD).
5.1	Cypress IDE	0x1080:0xc693	0x010100	IDE controller; fixed legacy ports (primary 0x1F0 / 0x3F6, secondary 0x170 / 0x376).
7.0	DE500 tulip	0x1011:0x0019	0x020000	21143 Ethernet; currently a generic stub (full model is planned).

The IDE controller carries two storage units on its primary channel: `dqa0` (primary master) is the bootable ATA fixed disk (raw flat image, 512-byte sectors, byte offset = LBA x 512), and `dqa1` (primary slave) is the ATAPI CD. Media for each is resolved from the manifest against the `[Storage] diskDir` in the ini; an empty media string means no disk / no disc.

IIC / FRU devices

The DS10 manifest also models the IIC (I2C) bus the firmware probes during GCT/FRU initialization: two system-status devices (0x70, 0x72), an SMB FRU EEPROM for the DS10 board (0xA2) and one for the EV6 CPU (0xA4), and a 256-byte RCM NVRAM (0xC0). The PCI vendor/device IDs and BAR sizes are marked provisional in the manifest pending a real hardware dump.

See Also.

[SuperIO Controller](#)⁶⁹

[Task Matrix / TimeOut Loops](#)⁷⁰

[Current Platform Support](#)¹⁰

9.2 Env Vars Operation Ref / Kill the EmulatR / Known Limitations /

The DS10 operational quick reference: where the environment-variable controls live, how to stop a running instance cleanly, and the limitations a DS10 operator should expect today. The full catalogues are on the dedicated reference pages; this page is the platform-scoped summary and the pointers.

Environment variables

EmulatR reads a set of `EMULATR_*` environment variables that override ini defaults -- the console TCP port (`EMULATR_CONSOLE_PORT`), the flash ROM path (`EMULATR_FLASH_ROM`), the idle fast-forward gate (`EMULATR_IDLEWARP`), trace directory, and others. The complete list

with semantics is the Environment Variables reference page; the SRM >>> variable catalogue (the firmware's own environment, set with `set`) is covered under Operating EmulatR.

Stopping an instance cleanly

Stop EmulatR by closing the console / GUI session so NVRAM and the flash ROM flush normally; avoid killing the process mid-write. The DS10 keeps its non-volatile state in `ds10_flash.rom` and the RCM NVRAM, so an interrupted write during a flash update can corrupt persisted firmware settings. When running two instances from one directory, isolate the flash with `EMULATR_FLASH_ROM` so concurrent NVRAM writes cannot collide.

Known limitations (DS10)

The SRM firmware boots all the way to the interactive >>> prompt; what is not yet complete is booting a guest operating system. No bootable disk is enumerated yet and the on-board PCI devices are not walked, and `b dqa1` currently reports "BOOT NOT POSSIBLE" (see Current Development Status).

The on-board DE500 Ethernet is a generic stub: the firmware tolerates the missing NIC, but it is not enumerated as a real tulip yet.

The `dqa1` CD is not always observed during the option-firmware scan (a known enumeration gap).

PCI vendor/device IDs and BAR sizes in the manifest are provisional pending a hardware dump.

See Also.

[Environment Variables \(EMULATR_*\)](#) 

[Executing Two Instances](#) 

[Current Development Status](#) 

9.3 SRM Boot / SRM Commands Prompt / SuperIO Controller /

The DS10 firmware reference. SRM boot and the >>> command set are covered operationally in the Quick Start and Operating EmulatR chapters; this appendix records the DS10-specific SuperIO analysis -- the part of the platform that is unique reference material rather than general operation.

SRM boot and commands (pointers)

Reaching the >>> prompt and the boot sequence are described under Quick Start; the console command catalogue (`show config`, `show device`, `boot`, `set`, and the rest) is under Operating EmulatR. The notes below assume that context and focus on the SuperIO programming the firmware performs during initialization.

SuperIO controller -- word-fold (confirmed)

The Cypress CY82C693 SuperIO is configured through 16-bit writes that fold an index byte and a data byte into one word: the low byte is the index, the high byte is the data. This byte order is locked and confirmed -- the 0x22 / 0x23 power-control writes carry their Table 61 default data bytes, and the configuration splits cleanly into `port 0x3F0` (index) / `port 0x3F1` (data):

```
SIO-TRACE W port=0x3F0 val=0x60 [cfg] index 0x60
SIO-TRACE W port=0x3F1 val=0x03 [cfg] data 0x03 (FDD base high)
SIO-TRACE W port=0x3F0 val=0x70 [cfg] index 0x70
SIO-TRACE W port=0x3F1 val=0x06 [cfg] data 0x06 (IRQ6)
```

Logical device map

The firmware programs every logical device in turn (selecting each via index 0x07 = LDN, then base / IRQ / DMA / activate). The resulting assignment matches `show config` exactly:

LDN	Device	Base	IRQ / DMA
0	Floppy (FDD)	0x03F0	IRQ6 / DMA2 (Activate set)
3	Parallel (LPT1)	0x03BC	IRQ7
4	Serial (COM1)	0x03F8	IRQ4
5	Serial (COM2)	0x02F8	IRQ3

The configuration is grounded end to end in the HRM trace: 93 SIO-TRACE config lines, every one splitting cleanly into index / data, with the FDD logical device landing at 0x03F0 / IRQ6 / DMA2. The SuperIO is properly modeled and the byte-fold is confirmed beyond doubt.

See Also.

[Appendix - DS10](#)⁶⁷

[show config](#)³²

[Task Matrix / TimeOut Loops](#)⁷⁰

9.4 Task Matrix / TimeOut Loops

Two pieces of DS10 engineering reference: the concrete bounds of the firmware's device-wait loops (useful when judging whether a long pause is a hang or a timed wait), and a snapshot of the open engineering work on the platform.

TimeOut loops

The firmware's IDE device-wait loops have concrete, bounded iteration counts. They are timed waits, not hangs; the longest is about half a minute of wall-clock per occurrence.

Loop function	Approx. wall clock	Notes
<code>dq_wait_on_busy</code>	5000 x 100 us	Wait for the IDE BSY bit to clear.

Loop function	Approx. wall clock	Notes
dq_wait_for_drq	5000 x 100--200 us	Wait for the data-request (DRQ) handshake.
dq_long_wait_on_busy	310000 x 100 us (about 31 s)	The long device-busy wait.

The `EMULATR_IDLEWARP` gate fast-forwards these idle waits without altering committed time. A separate, much longer pause -- the `kern$_sleep`-based option-firmware scan after "Checking dva0.0.0.0 for the option firmware files. . ." -- is a timed sleep rather than a device-wait loop and is a known performance item.

Task matrix (engineering status)

A snapshot of open DS10 engineering work, grouped by theme. This is a moving target; treat it as orientation, not a contract.

Networking. Promote the DE500 from a generic stub to a real DC21143 (tulip) model: enumerate ewa0 with a MAC read over the CSR9 SROM bit-bang, then add link / media select (SIA/MII) and the TX/RX rings for network boot.

Storage. Wire the media-backed read path (ATA READ 0x20 + ATAPI READ(10) to the file backend) so booting dqa0 works, and close the absent-unit IDE taskfile-read behavior.

PCI enumeration. Implement a real PCI bus walk so on-board devices are discovered and given valid BARs, replacing the firmware's all-ones fallback (the source of the repeated unhandled-outer-write probes).

Performance. Fast-forward the option-firmware scan sleep; keep the chipset off the hot path with a DRAM fast-path for fetch and load/store; event-driven console RX instead of a per-step poll; compile-guarded trace hooks.

Operational. Make system memory size resolvable from the QSettings ini; neutralize the "Halt Button is IN, BOOT NOT POSSIBLE" gate in software; centralize trace / diagnostic output under the build directory.

See Also.

[Firmware Hangs, Slow Boot, and Missing Devices](#)⁶²

[Halt Button is IN, BOOT NOT POSSIBLE](#)⁶³

[Current Development Status](#)¹¹

9.5 Appendix DS20 / ES40 / ES45

The larger Tsunami-family machines are planned but not yet modeled or validated in EmulatR. They are recorded here so the platform roadmap is explicit; none of them currently reaches the >>> prompt.

DS20

Planned. DS20 is not yet modeled or validated. A platform manifest and reference notes will be added once the model exists.

ES40

Planned. The platform manifest `es40_platform.json` is scaffolded, but the platform has not yet been driven to the >>> prompt. To be validated and documented.

ES45

Planned. The platform manifest `es45_platform.json` (EV67 / 21264A) is scaffolded but not yet validated. To be documented once it boots.

Note that the compressed firmware oracle work (`host_decompressor`) was developed against `es45_v7_3.exe`, so the ES45 firmware image decompresses correctly even though the full platform is not yet driven; what remains is the device and chipset modeling around it.

See Also.

[Appendix - DS10](#) 

[Current Platform Support](#) 

[Decompressor Internals](#) 

10 Subsystem Internal

Developer-reference notes on individual subsystems: how floating point is approached, how the grain dispatch machinery is generated from tables, and how build-time behavior is gated by compile guards. These complement the operator-lens Architecture & Internals chapter and the register-level deep-dives in 8.3.

In this section

- *Floating-Point Considerations* -- the four formats, the FP-enable gate, FPCR, and the backend strategy.
- *grainFactoryLib* -- the table-driven grain codegen.
- *DispatchTables* -- the primary and per-kind dispatch tables that bind an opcode to a leaf.
- *CMake Macro Guards* -- the build-time options and diagnostic compile guards.

See Also.

[Architecture Deep-Dives](#)⁷⁹

[Architecture Overview](#)⁴⁶

10.1 Floating-Point Considerations

Alpha floating point is wider than a single IEEE implementation, and modeling it faithfully has several moving parts: four data formats, a separate enable gate, a control register that selects rounding and trap behavior, and a choice of compute backend. This page records the considerations a developer needs when reading or extending the fBox; it is a design overview rather than a per-opcode status table, because the instruction coverage is actively being filled in.

Four formats

Alpha supports two IEEE formats -- S (single) and T (double) -- and the VAX formats F, G, and D. The distinction matters for guests: OpenVMS Alpha uses VAX G_float as its default double and F_float as its default single, while Tru64 and IEEE workloads use S and T. Data movement is format-complete: the loads and stores for all four memory formats (LDF/LDG/LDS/LDT and STF/STG/STS/STT) move values correctly, so a guest can hold VAX G_float values in registers. The work that distinguishes the formats is the arithmetic and the conversions between them, which is where build-out effort concentrates.

The FP-enable gate

Floating point is gated by the FEN (FP-enable) bit. The PAL path for it is in place: MTPR_FEN and MFPR_FEN move the bit, CLRFEN clears it, and an attempt to execute FP while disabled delivers through the FEN entry vector at offset 0x200. So the enable / disabled-trap machinery works independently of how much arithmetic is implemented behind it.

FPCR, rounding, and traps

The floating-point control register (FPCR) selects the dynamic rounding mode and the IEEE trap-enable bits; instruction qualifiers select trap mode and rounding per-instruction. Faithful FP must honor these rather than always rounding to nearest and never signaling. The implementation approach uses the host's `<cfenv>` facilities -- `fesetround`, `feclearexcept`, `fetestexcept` -- around a native host operation to realize the FPCR rounding mode and to detect and deliver IEEE exceptions. Move-to / move-from FPCR provide the storage; the qualifier handling is what connects it to each operation.

The backend abstraction

FP arithmetic is reached through a backend interface, `IFpBackend` (in `fpBoxLib`), so the grain leaves call an abstract operation set -- the IEEE S/T and VAX F/G add / subtract / multiply / divide / square-root, the compares, and the format conversions -- without binding to one implementation. Operands and results are passed as raw register images (`uint64_t`) plus an `FpExecCtx` that carries the rounding mode and qualifiers, and an `FpResult` that carries the value and the exception flags. Wiring this interface into every FP grain is the in-progress integration step.

Berkeley SoftFloat -- the deterministic oracle

The reference implementation of `IFpBackend` is `SoftFloatBackend`, built on the Berkeley SoftFloat Release 3 library (vendored in the tree as `berkeley-softfloat-3-master` and compiled into a `softfloat` static library, 8086-SSE specialization). It is the deterministic, bit-identical-across-hosts FP truth source: the rounding mode is passed to SoftFloat as data and the exception flags are harvested as data, so no host control word (MXCSR / FPSR) is ever touched and the result is identical on every host and every build. `SoftFloatBackend` implements the full surface, IEEE and VAX alike, including the VAX F/G arithmetic, compares, and conversions that OpenVMS depends on.

An optional native fast-path backend (`HostBackend`) provides the seam for using host FP hardware where it is faster; it inherits `SoftFloatBackend` so any operation it has not specialized still delegates to the deterministic path, and its results are validated against SoftFloat (the `fp_softfloat_vs_host_bench` test exists for exactly this). The build-time `EMULATR_FP_SOFTFLOAT` option selects SoftFloat as the active backend; the longer-term aim is to expose the SoftFloat-versus-native choice as a runtime setting. Where a leaf still uses the host directly, the FPCR rounding and IEEE-trap behavior is realized with `<cfenv>` as described above; the SoftFloat path needs no such host-state juggling because it carries rounding and flags as data.

See Also.

[grainFactoryLib](#)⁷⁵

[CMake Macro Guards](#)⁷⁶

[Attributions & Credits](#)  59

10.2 grainFactoryLib

A grain is the unit of instruction semantics in V4: each decoded instruction resolves to a single grain leaf that implements its behavior. grainFactoryLib is where those leaves and the tables that reach them are generated, from tab-separated source-of-truth tables rather than hand-maintained switch statements.

Table-driven codegen

The codegen reads two tables and emits generated C++ headers and sources.

`SemanticFlags.tsv` defines the semantic flag vocabulary; `GrainMasterV4.tsv` is the per-instruction master table. The generator `codegen/genGrains.py` validates the join between them and emits, under `generated/`: `SemanticFlagsEnum.h` (the flag enum, masks, and predicates), `DispatchKinds.h` (the dispatch-kind enum), `GrainsForward.h` (per-box leaf declarations), `DispatchTables.cpp` (the populated tables), and `GrainStubs.cpp` (stub leaves for not-yet-implemented rows). Hand-written leaves are carried in `codegen/handwritten.tsv` so a real implementation can supersede a generated stub.

The master table row

Each row of `GrainMasterV4.tsv` describes one instruction with these columns: `mnemonic`, `primaryOpcode`, `subDecode`, `dispatchKind`, `format`, `box` (the functional domain that owns the leaf -- IBox, EBox, FBox, MBox, CBox, or PalBox), `flags` (a bitwise-OR of `S_*` semantic flags), `notes`, and `leafBase`. The flags drive correctness-relevant behavior: for example a PAL row carries `S_PalFormat` | `S_PalEntry` | `S_ChangesPC` and one or more personality flags.

PALcode personalities

A single CALL_PAL opcode can mean different things under different operating systems, so rows carry personality flags -- `S_PalTru64`, `S_PalVms`, and `S_PalLinux`. For instance opcode 0x83 is CHMK (change mode to kernel) under VMS but callsys under Tru64 and Linux; the row marks all three. The codegen iterates the personalities to emit the right dispatch, which keeps the OS-specific PAL behavior in the table rather than in code.

See Also.

[DispatchTables](#)  75

[Execution Flight Plan](#)  52

[Floating-Point Considerations](#)  73

10.3 DispatchTables

The dispatch tables are how the Decode stage turns a raw instruction word into a grain leaf. They are generated into `DispatchTables.cpp` from the master table, so the decode path is data, not a hand-written switch.

Primary and secondary tables

The top-level lookup is `g_primaryTable`, indexed by the 6-bit primary opcode. Many Alpha opcodes are not fully decoded by that field alone -- arithmetic, floating point, the `HW_MxPR` instructions, and the PAL group all carry a sub-function in lower bits. For those, the primary entry points at a secondary (sparse) table keyed by the row's `subDecode` value, so the full decode is a primary index followed, where needed, by a sparse secondary lookup. The Decode stage performs exactly that: read the opcode, index the primary table, and follow the sub-decode where the kind requires it, yielding the grain leaf that the Execute stage will invoke.

Dispatch kinds

Each instruction is classified by a `DispatchKind` that names its decode family and selects which secondary table (if any) applies. The canonical set covers the integer families `IntArith`, `IntLogical`, `IntMul`, and `IntShift`; the floating-point families `FltIeee`, `FltVax`, `FltLogical`, `ItFp`, and `FpTiExt`; the privileged / hardware kinds `HwLd`, `HwSt`, `HwMfpr`, `HwMtpr`, `HwRei`, and `Pal`; and the control / miscellaneous kinds `Direct`, `JumpClass`, and `Misc`, plus a `Reserved` sentinel. A name helper (`dispatchKindName()`) is generated alongside the enum for trace and diagnostics.

Because the tables are generated, adding or correcting an instruction is a table edit plus a regeneration, not a hand change to decode logic -- the same source-of-truth discipline as the rest of `grainFactoryLib`. A row with no implemented leaf resolves to a stub (which logs and does not compute) or, if there is no row at all, the opcode takes a decode fault (OPCDEC).

See Also.

[grainFactoryLib](#) ⁷⁵

[Execution / Boot Internals](#) ⁸⁹

[Execution Flight Plan](#) ⁵²

10.4 CMake Macro Guards

A number of EmulatR behaviors are selected at build time through CMake options that define compile-guard macros, rather than at run time. This keeps diagnostic and debug scaffolding out of the shipping hot path: a guard that is off compiles to nothing. The options are set in the CMake configuration (for example through Qt Creator) and define the matching `EMULATR_*` macro for the build.

Build / behavior options

Guard	Effect
<code>EMULATR_DEBUG_STEP</code>	Restores an -O0 build of the per-instruction pipeline step for

Guard	Effect
	single-step debugging. Off by default so the hot loop ships optimized.
EMULATR_FP_SOFTFLOAT	Selects the SoftFloat compute backend for floating point instead of the native host path, for bit-exact host-independent results.
EMULATR_USE_OS_PAGES	Selects the OS-page (VirtualAlloc / mmap) backing mode for guest memory.

Diagnostic / trace guards

Guard	Effect
EMULATR_TRACE_HOOKS	Compiles in the per-step trace hooks used to capture an execution trace.
EMULATR_MEMDIAG	Compiles in the memory-diagnostic instrumentation (the cycle-windowed fetch / load / store dumps).
EMULATR_CHIPSET_DIAG	Enables chipset-level diagnostic output.
EMULATR_DIAGNOSTIC_LOGGING	Enables the broader diagnostic logging path.
EMULATR_PA_DUMP	Compiles in physical-address dump instrumentation.
EMULATR_BOOTSTRAP_ITB_BYPASS	Debug only: bypasses the ITB for the known reset / PAL entry page during bring-up. Must be removed before relying on permission bits or ASN isolation.
EMULATR_TRACE_CONSOLE / EMULATR_ISADIAG	Console-trace and ISA-diagnostic channels.

The default build leaves the diagnostic guards off; they are turned on only when chasing a specific problem, because the instrumentation they add (especially the memory and trace hooks) is heavy enough to change runtime substantially. Runtime behavior that does not need a recompile is controlled instead by the `EMULATR_*` environment variables.

See Also.

[Building from Source](#)  14

[Qt Creator](#)  93

[Environment Variables \(EMULATR_*\)](#)  55

11 Architecture Deep-Dives

These appendices are the register-and-address-level companions to chapter 5 (Architecture & Internals). Where chapter 5 explains a subsystem for an operator, the deep-dives name the V4 classes, the numeric IPR indices, the entry-vector offsets, and the source files -- the level a developer needs to read or extend the code.

In this section

- *Memory Map & IPRs -- the physical address regions and the internal processor register space.*
- *Addressing & TLB -- the Ev6 translator, the SPAM shard manager, and TLB epoch invalidation.*
- *Decompressor Internals -- the DEC inflate sources and the host_decompressor oracle.*
- *Execution / Boot Internals -- the six-stage pipeline and fault entry vectors.*

See Also.

[Architecture Overview](#) 

[Appendices](#) 

11.1 Introduction & Overview

EmulatR is a full-system, high-fidelity emulator of the DEC Alpha AXP platform -- specifically the 21264 (EV6) processor and the 21272 (Tsunami / Typhoon) core logic as integrated in AlphaServer DS10 / ES40 / ES45-class systems. It is implemented in modern C++ (C++20) with a deliberately framework-free architectural core (Qt is confined to the periphery -- console UI and host glue), built with CMake. Its purpose is not to approximate an Alpha; it is to be one, faithfully enough that unmodified DEC SRM console firmware, PALcode, and the operating systems that ran on the metal run on EmulatR without knowing the difference.

This page describes the architecture of the current implementation (V4) -- the execution model, the memory and translation system, the privileged boundary, and the device fabric -- and the standard every subsystem is measured against. The chapters that follow expand each area; the deep-dive appendices carry the register-level detail.

The fidelity standard and its authoritative sources

EmulatR is governed by one standard: it must behave like real Alpha hardware in every architecturally visible way, even when that complicates the implementation. Correctness is defined by primary sources -- the **Alpha Architecture Reference Manual** and the original **DEC SRM console and PALcode sources**. These define what correct behavior is.

The test is black-box and adversarial, conducted by the most demanding guests available: the SRM console and PALcode exercise the privileged architecture directly, and a booting operating system exercises the memory model, the trap machinery, and the device fabric under real workloads. A second emulator (AXPBox) is consulted only as a convenience cross-check for differential triage -- it is explicitly **not** an authoritative source, because another implementation can be wrong, whereas the architecture manual and the firmware sources cannot. If a guest can distinguish EmulatR from an AlphaServer through architecturally defined behavior, the emulator is wrong.

Primary goals and objectives

1. **Architectural correctness first.** EmulatR models Alpha's architectural semantics, not a particular chip's timing. Correctness always outranks speed.
2. **Deterministic, debuggable execution.** Execution is reproducible and inspectable. The run loop is staged and cycle-counted, and the hot path is deliberately shallow so a stack trace at any point stays short and legible.
3. **Explicit, per-instruction semantics.** Every instruction resolves to one faithful grain leaf; behavior is never inferred from an operand-shape family. Architectural results and exceptions become visible only at the Writeback stage.
4. **Separation of concerns.** Decode, integer, floating-point, memory, control, and privilege are isolated into well-defined functional domains -- the "Boxes" -- with one-way dependency flow. No domain reaches into another's internal state.
5. **SMP-aware by construction.** Per-CPU state, CPU-local translation buffers, and load-locked / store-conditional reservation tracking are built in, so a multiprocessor build is an extension rather than a rewrite (see the scope note below).

The execution model

Execution is driven by `PipelineDriver` (`pipelineLib`) through six in-order stages: `IF` (instruction fetch), `DE` (decode / dispatch), `GR` (grain-ready: read the register file into the execution context), `EX` (execute), `MEM` (memory drain), and `WB` (writeback). Each instruction is carried through the six stages on a `PipelineSlot`; architectural state -- the register file and the PC -- is committed only at `WB`, the single well-defined commit point that makes exceptions precise and replay deterministic.

Dispatch is flat. At `DE` a decoded instruction is resolved through the grain system to a single grain leaf -- a pure function with the signature `BoxResult(InstructionGrain const&, ExecCtx const&)` -- that implements its semantics. Execution is partitioned into the functional Boxes, each a namespace of grain leaves rather than a class hierarchy: **IBox** (control flow -- branches, jumps), **EBox** (integer ALU), **FBox** (floating point, IEEE and VAX, over the `fpBox` backend), **MBox** (loads, stores, load-locked / store-conditional), **CBox** (branch prediction and target resolution), and **PaiBox** (the privileged boundary). Dependencies flow one way: every Box depends on `coreLib`; most also on `iprLib`; `MBox` additionally on `memoryLib`.

A deliberate consequence is that the hot path is shallow -- from the driver, through the stage, into a grain leaf, the call-stack depth is typically two and rarely exceeds four. That keeps traces legible (serving debuggability) and minimizes the per-instruction scaffolding that profiling has shown -- rather than the opcode bodies -- to dominate runtime (serving performance without compromising correctness). The grain leaves are the leaves of that call tree by construction.

Dispatch tables are generated, not hand-maintained. `GrainMasterV4.tsv` is the single source of truth for instruction semantics; the `genGrains.py` codegen reads it and emits the dense primary table (indexed by the 6-bit opcode) plus the sparse secondary tables for multi-function opcodes, including the personality-aware PALcode dispatch (OSF/1 and VMS). There is no runtime trace cache or JIT; the generated tables are the decode artifact.

Memory and address translation

Physical backing. `GuestMemory` (`memoryLib`) is the single physical-memory interface -- a passive byte-store that pages the guest's physical address space in on demand, in 64 KiB pages, committed only as the guest first touches them, so a multi-gigabyte machine costs only its working set. The pages are backed by host `VirtualAlloc` on Windows and by `mmap` on Linux / ARM hosts (with a portable fallback). The earlier `SafeMemory` layer has been removed. `GuestMemory` itself does no MMIO routing or fault generation: the `TsunamiChipset` system-bus arbiter decodes addresses, routes device traffic, and raises range faults before raw RAM is ever touched. A `LockMonitor` tracks per-CPU load-locked / store-conditional reservations at cache-line granularity.

Guest translation. Address translation is performed by `Ev6Translator` (`mmuLib`) as the EV6 architecture specifies: PAL-mode bypass, the canonical virtual-address window check, and superpage (kseg) mapping, with non-mapped accesses returning an ITB or DTB miss. The translation buffers are modeled by the **SPAM** cache -- Set-Partitioned Associative Memory -- as `SPAMShardManager` instances resident in `CpuState` (`itbMgr` and `dtbMgr`, each 16 shards by 8 ways = 128 entries). Invalidation is epoch-based and lazy (`TlbEpoch`): a TBIA bumps a global counter and a TBIAP a per-process one, so a flush is O(1) and stale entries simply fail the liveness test at lookup. The translation buffers are CPU-local by design.

Faults, exceptions, and the privileged boundary

There is no central fault dispatcher. A leaf, or the memory drainer that detects a fault, sets a `faultCode` on the instruction's `BoxResult` (for example a DTB miss or an unaligned access). The hardware exception entry points are compile-time constants in `Ev6EntryVectors.h` (the vectors at `0x100` through `0x780`, spaced `0x80` apart). At WB the driver inspects the fault: a halt sets the CPU's halted flag; otherwise, once `PAL_BASE` is established, control diverts to the computed entry vector. Privileged operations -- `CALL_PAL`, `HW_MTPR` / `HW_MFPR`, `HW_REI`

-- are handled in `PalBox`, which applies IPR side-effects (PAL_BASE normalization, TLB invalidation, ASN context) inline. Internal processor registers are flat members of `CpuState`.

The machine and its device fabric

`Machine` (`systemLib`) is the top-level orchestrator: it owns the `CpuState`, the `GuestMemory`, and the `TsunamiChipset`, and provides firmware load, reset, and the step / run entry points. The chipset aggregates the three 21272 components -- the **Cchip** (system control, memory arbitration, interval timers, DRAM array configuration), the **Dchip** (data path), and the **Pchip** (PCI host bridge) -- together with the on-board devices. The PCI fabric is a standard bus / slot / function hierarchy (there is no separate "stratum" abstraction).

The southbridge is the **Cypress CY82C693** PCI-to-ISA bridge (`Cypress_CY82C693ISABridge`), which carries the ISA / SuperIO plane. The modeled devices include the CY82C693 IDE / ATAPI channel, the SuperIO controller (FDC / COM / LPT), the TOY / RTC, a 16550 UART for the serial console, the PCF8584 IIC controller, the flash / SRAM, and the dual 8259A interrupt controllers (`Pic8259Pair`). SCSI endpoints (`VirtualScsiDevice` and the ISO-CD variant) exist in the device library.

Lineage: four implementations, one set of objectives

The current emulator is the fourth implementation; the objectives above were fixed from the start, and each version reached for them by a different mechanism.

- *V1 dispatched each instruction through a virtual call to a per-instruction grain. It was correct and booted SRM, but the per-call overhead capped throughput.*
- *V2 chased performance by dispatching on instruction families (operand-shape signatures). It was fast, but families under-determined behavior -- instructions sharing a signature still needed per-instruction corner-case handling the model could not express.*
- *V3 tried to hybridize V1 and V2; the coupling was too tight and introduced silent-failure modes -- the worst outcome for an emulator whose value is fidelity.*
- *V4 keeps a flat dispatch path but resolves it to faithful per-instruction grain leaves via generated tables, with performance pursued as the sum of many marginal gains rather than one architectural bet. The dispatch mechanism evolved across all four; the objectives did not.*

Scope note: this page describes the V4 architecture and the standard it is held to. Some elements are infrastructure ahead of full activation -- multiprocessor execution and cross-CPU IPI, full PAL trap delivery, and SCSI / PCI device enumeration. For what is implemented and working in the current build versus what is still in progress, see [Current Development Status](#).

See Also.

[Memory](#) 

[Addressing](#) 

[Execution Flight Plan](#) 

[Current Development Status](#)  ¹¹

11.2 Memory Map & IPRs

This appendix is the register-level companion to the Memory and Addressing pages. It names the two internal-processor-register (IPR) namespaces EmulatR implements, the numeric indices a developer reads in a trace, and the physical-address regions the chipset arbiter routes. The authoritative reference is the Alpha 21264 Hardware Reference Manual (EC-RE2CA-TE, Table 5-4) together with V4's `coreLib/HW_IPR.h`.

Two IPR namespaces

EmulatR follows the architecture's split between registers the operating system reaches through `CALL_PAL` and registers only PALcode can touch with `HW_MFPR` / `HW_MTPR`.

1. PALcode-visible IPRs (0x0000 .. 0x00FF). These are `CALL_PAL` function codes, not hardware registers. The OS executes a `CALL_PAL` (for example `MFPR_PTBR` = 0x0015, `MTPR_TBIA` = 0x001B, `MFPR_WHAMI` = 0x003F); the PAL handler at the matching entry vector performs the actual access. Numbering follows the OSF/1 PALcode specification rather than the silicon. In V4 these are the `PAL_MFPR` / `PAL_MTPR` enums.

2. EV6 hardware IPRs (0x0100 .. 0x02FF). These are the real register indices encoded in the `scbd` field of an `HW_MFPR` / `HW_MTPR` instruction; only PALcode running in PAL mode may execute them. Numbering follows the 21264 HRM. The 0x0100 offset is an emulator convention so the hardware range cannot collide numerically with the PALcode-visible range: the decoder adds 0x0100 to the raw `scbd` value, except the `PAL_TEMP` block (raw 0x40..0x5F) which adds 0x01C0 so `PT0` lands at 0x0200.

Hardware IPR map (the indices you see in a trace)

The hardware IPRs are grouped by the box that owns them. Selected indices from

`HW_IPR.h`:

Index	Name	Box / role
0x0106	<code>HW_EXC_ADDR</code>	IBox -- exception address; read = saved PC, write = <code>HW_REI</code> target. Bit[0] carries PAL mode.
0x0110	<code>HW_PAL_BASE</code>	IBox -- PAL base address (32K-aligned; low 15 bits zero).
0x0111	<code>HW_I_CTL</code>	IBox -- IBox control register (SDE and related bits).

Index	Name	Box / role
0x0113	HW_IC_FLUSH	IBox -- I-cache flush all (action).
0x0120 / 0x0121	HW_DTB_TAG0 / HW_DTB_PTE0	MBox -- DTB tag staging and PTE fill, bank 0 (writing the PTE triggers the fill).
0x0127	HW_MM_STAT	MBox -- memory-management status (read-only; fault detail).
0x0140	HW_PCTX	Process context -- ASN + FPE + PPCE packed.
0x01C0 / 0x01C1	HW_CC / HW_CC_CTL	Cycle counter and its control (offset + enable).
0x01C2 / 0x01C4	HW_VA / HW_VA_CTL	Faulting virtual address (read-only) and VA control (write-only, packed -- selects the 43/48-bit VA window).
0x01A0 .. 0x01A5	HW_DTB_*1	MBox -- DTB bank 1 (the EV6 dual-bank DTB).
0x0200 .. 0x0217	HW_PAL_TEMP_0 .. _23	PAL temporaries PT0..PT23 (PAL-mode scratch; OSF/1 assigns KSP, PTBR, PCBB, etc.).
0x0218 .. 0x021F	HW_PAL_TEMP_24 .. _31	PT24..PT31 -- EV5-vintage scratch, provisioned because bring-up SRM .exe code addresses PT31.

PAL temporaries

The EV6 provides 24 PAL temporary registers (PT0..PT23), reachable only via HW_MFPR / HW_MTPR in PAL mode. They are the backbone of PALcode: every PAL entry saves the guest GPRs into PAL_TEMP_s and every PAL exit restores them. The OSF/1 convention pins specific temporaries to architectural state exposed through CALL_PAL (PT0 = KSP, PT2 = PTBR, PT4 = PCBB, PT5 = PRBR, PT6 = SCBB, and so on). The OS never reads a PAL_TEMP directly -- it issues CALL_PAL MFPR / MTPR and the handler relays to the right temporary. V4 widens the range to PT31 because the SRM firmware used during bring-up carries EV5-

vintage code that addresses PT24..PT31 even on EV6 (observed at PC 0x12f88 issuing `hw_mtpcr Rx, 0x5F`); those extra eight are pure CpuState-backed scratch.

IPRs that map to architectural state live as flat fields on `CpuState` (`ptbr`, `i_ctl`, `pal_base`, `ps`, `excAddr`, `fpcr`). Selectors that are not yet CpuState-backed return `kFaultUnimplemented`; adding one later is a CpuState field plus a switch case in the HW_MFPR / HW_MTPR leaf.

Physical address regions

Guest physical addresses are routed by the Tsunami chipset arbiter. Reads and writes resolve to one of three destinations: main memory (the GuestMemory sparse backing, bounded by the Cchip AAR registers -- anything outside is a non-existent-memory / NXM fault), the chipset CSR space (Cchip / Dchip / Pchip control registers and the TIG region), or PCI / device MMIO windows. The halt-gate diagnostics read and write CSRs in this space; for example the console arming read targets `pmem:80130000FF8` in the TIG region. The exhaustive per-register CSR table is in the Tsunami / 21272 Hardware Reference Manual; the operator-facing view of how to read these locations is on the Evaluate Address page.

See Also.

[Memory](#)  50

[Addressing & TLB](#)  85

[Evaluate Address](#)  64

11.3 Addressing & TLB

This appendix is the implementation-level companion to the Addressing page. It walks the V4 translation path from a virtual address to a physical address, then describes the translation-buffer front end and the epoch scheme that makes a TLB flush a single counter bump. The authoritative reference is the Alpha Architecture Reference Manual (sections 4.1.6 / 4.1.7) and the 21264 Hardware Reference Manual; the V4 code lives in `mmuLib/Ev6Translator.h` and `pteLib/`.

The translator

`Ev6Translator` is not an object -- its entry points are static functions that take a `CpuState const&` as the first argument, so per-CPU state travels through `CpuState` rather than a translator instance. The MEM stage calls one of three entry points depending on the memory effect a leaf packed onto its `BoxResult` at EX:

- *translateData* -- a data load or store, no alignment check.
- *translateDataAligned* -- a data load or store with a size-class alignment check (1/2/4/8 bytes).
- *translateInstruction* -- an instruction fetch (longword-aligned, consults the I-side superpage enables).

Each returns a `TranslationResult`. On Success the caller's `pa_out` holds the physical address; any other value is a translation fault that the MEM drainer converts to a fault code (`mmuLib::toFaultCode`) and uses to short-circuit the register-file commit.

Translation path (data stream)

For a normal load or store, `translateData` proceeds in this order:

1. **-Explicitly-physical accesses bypass upstream.** HW_LD / HW_ST / LDQP / STQP are tagged physical and the MEM drainer sets `pa = va` before ever calling the translator. Importantly, EV6 PAL mode does NOT blanket-disable D-stream translation -- a normal load or store in PAL mode still translates. (An earlier blanket PAL bypass mis-mapped VMS system-space kernel-stack addresses and was removed.)
2. **-Canonical VA check.** VA_CTL bit 1 selects 48-bit VA mode when set, otherwise 43-bit mode. In 43-bit mode VA<63:43> must sign-extend VA<42>; in 48-bit mode VA<63:48> must sign-extend VA<47>. A non-canonical address faults before any TLB lookup.
3. **-Kseg / superpage detection.** The three SPE bits (on M_CTL for data, I_CTL for instructions) gate three kernel-only superpage modes. SPE[2] maps VA<43:13> directly to PA when VA<47:46> = 0b10; SPE[1] and SPE[0] cover the other two windows. A kseg-shaped address from non-kernel mode is an access violation; a non-kseg address falls through.
4. **-DTB lookup.** On a live hit, the PTE permission bits are checked and PA is composed as `(PFN << 13) | (VA & 0x1FFF)`. On a miss the result is `DtbMiss`, which the drainer maps to `kFaultDtbMiss`; PALcode's DTB-miss handler refills the buffer (HW_MTPR DTB_TAG0 / DTB_PTE0) and the access retries.

The instruction path mirrors this with a 4-byte alignment check (PC bit 0 is the PALmode flag and is masked before the check), the I-side SPE bits, and an ITB lookup that misses to `kFaultItbMiss`. The EV6 base page is 8 KiB (shift 13); a PA is 44 bits wide.

The translation buffers (SPAM)

Each `CpuState` owns its own pair of `SPAMShardManager` front ends -- one ITB, one DTB. A manager holds an array of `SPAMBucket` shards, the epoch state, and a small hash that picks a shard from the tag. In `CpuState` the two managers are instantiated as `SPAMShardManager<16, 8>` -- 16 shards by 8 ways = 128 entries each, matching the architectural EV6 ITB and DTB size (the shard count and ways are template parameters, so the geometry can be retuned without touching the lookup path). Its public surface mirrors the architectural operations: `lookup` (one per access), `insert` (the miss handler / HW_MTPR ITB_PTE / DTB_PTE), the three invalidate operations, and `clear` for cold boot. Because translation buffers are CPU-local by construction, the manager carries no synchronization primitives; any future SMP cross-CPU invalidation would arrive through IPI machinery processed on the target CPU's own thread.

Epoch invalidation

A naive TLB would walk every entry on a flush. V4 instead keeps two counters in the manager and stamps both into every entry at insert time. A flush of all entries is then a single counter bump; entries become stale the instant their stamped value fails to match the live counter on the next lookup. The mapping from PALcode flush operations is:

Operation	Action	Lazy?
TBIA	bump globalEpoch (invalidates everything)	yes
TBIAP	bump processEpoch (ASM=1 global entries survive)	yes
TBIS / TBISI / TBISD	locate the matching entry, clear it in place	no (targeted)

The liveness test applied at lookup is: an entry is live when its globalEpoch matches the manager's globalEpoch AND either the entry is ASM-global or its processEpoch matches. The ASM arm encodes the architecture's rule that an ASM=1 (global) mapping survives TBIAP. The counters are plain 64-bit integers, not atomics -- at one bump per 100 ns a 64-bit counter takes roughly 58,000 years to wrap, so wrap is not handled; a manager reset zeroes both counters and clears every shard if it ever matters.

See Also.

[Addressing](#) ⁵⁰

[Memory Map & IPRs](#) ⁸³

[Execution / Boot Internals](#) ⁸⁹

11.4 Decompressor Internals

This appendix is the source-level companion to the Firmware Decode & Decompression page. It identifies the DEC sources behind the cold-boot self-decompression, the CPU/PAL side-effects the guest startup performs, and the native oracle EmulatR uses to prove its emulated CPU produces the correct image. Treat the DEC SRM sources as authoritative; AXPBox is not.

What runs at cold boot

The guest firmware ships compressed and decompresses itself on the emulated CPU every cold boot -- roughly 4 million cycles, visible as the spin at PC `0x60111c`. The image carries a `WimC` header (magic `0x436D6957`) located by a linear scan; its longwords give the compressed size, the decompress target, and the start of the compressed data. For `es45_v7_3.exe` the header sits at file offset `0x2400`, the target is `0x8000`, and the decompressed output is `0x30cc00` bytes (3,197,952).

The DEC sources

The read-only originals live in `Processor`
`Support\Palcode\palcode\apisrm\apisrm\ref\`:

- *inflate.c* -- the engine: Mark Adler inflate, version c10p1.
- *decom.c* -- the DEC wrapper providing `main()` and `decompress()`.
- *decomp.h* -- types and the ROM / non-ROM switches.
- *ev6_huf_decom.m64* -- the guest assembly startup that calls `decompress()`.

The startup is more than a memcpy

The `ev6_huf_decom.m64` startup is not a pure decompression: before and around the call it performs real CPU/PAL state setup -- it loads `PAL_BASE`, sets the `I_CTL` SDE bits, flushes the ITB / DTB and the I-cache, manages shadow registers, and saves and restores the SROM parameters in R16-R21. Any alternate "host decompress" path must preserve that end state, which is why a full bypass is risky and the recommended future design is an intercept-hybrid rather than a wholesale skip.

The native oracle

`tools/host_decompressor/` builds the SAME algorithm DEC used, from the SAME sources the firmware was compiled from, as a native host program with no emulation involved. Its harness `oracle.c` replaces `decom.c`'s `main()`: it finds the WimC header, sets the input/output pointers and compressed size, calls `inflate()`, and dumps the image to `ref_decompressed.bin`. The output matches EmulatR's emulated-CPU decompression byte for byte, which makes it a trusted regression guard. This is strictly stronger than AXPBox's `decompressed.rom`, which is produced by running the guest decompressor on AXPBox's own emulated CPU (and so still depends on a correct CPU and a contrived entry sequence).

The oracle settled the "single-bit PAL corruption" question: the instruction at PC `0xd954` (encoding `0x7be2a000 = hw_ret R2`) is genuine firmware, not a decompression artifact. The known-good signature is the quad at PA `0xd950` -- a value using `R2 (0xe2)`, never `R6 (0xe6)`. If a future change makes the guest CPU produce `R6` there, that is a CPU regression, not a fix.

Deferred: a host-native boot path

Integrating the oracle as a runtime alternate path (`--decompress=inline|host|cache`, default `inline`) is deferred until the SRM console reaches the `>>>` prompt. The recommended design is the intercept-hybrid: let the guest huf assembly run so its CPU/PAL side-effects happen for real, intercept entry to the `decompress()` C function, host-fill the output memory, set `R0` to the decompressed base, and advance PC to the call's return site -- skipping only the multi-million-cycle inner inflate while inheriting every side-effect. A full

bypass risks a subtle missing-side-effect divergence that would be painful to chase because the image bytes look perfect.

See Also.

[Firmware Decode & Decompression](#) ⁵¹

[Execution / Boot Internals](#) ⁸⁹

11.5 Execution / Boot Internals

This appendix is the implementation-level companion to the Execution Flight Plan. It walks one instruction through V4's six pipeline stages, then describes how a fault is delivered to PALcode and how the firmware boot path is fed during the decompression loops. The code is in `pipelineLib/PipelineDriver.h`; the entry-vector table is in `coreLib/Ev6EntryVectors.h`.

The six-stage step

EmulatR runs a functional, single-slot pipeline: each instruction is carried by a `PipelineSlot` through all six stages in order before the next is fetched. The pipeline driver is the only component that mutates `cpu.pc` -- the leaves and the memory drainer never touch it.

Stage	Work
IF	Mint a slot from <code>cpu.pc</code> ; translate the VA to a PA (via <code>translateInstruction</code>) and read the 4-byte instruction word. The fetch consults an optional <code>IFetchOverride</code> first.
DE	Dispatch lookup against <code>grainFactory::g_primaryTable</code> (plus the sparse secondary tables) to bind the decoded grain leaf.
GR	Build the <code>ExecCtx</code> : read the register file (integer or FP per the grain) into the execution context.
EX	Invoke <code>grain.execFn(grain, ctx)</code> and store the returned <code>BoxResult</code> onto the slot (any memory effect is packed here, not performed).
MEM	<code>MemDrainer::drain(slot, cpu, memory)</code> -- translate any data effect, perform the load/store, sign-extend loads, commit the register file, and resolve LL/SC.
WB	Advance <code>cpu.pc</code> from the grain's next PC, a divert target, or a fault entry vector;

Stage	Work
	intercept HALT and faults; increment the cycle count.

The driver exposes `step` (one slot through all six stages) and `run(maxCycles)`. The hot loop ships optimized; an `EMULATR_DEBUG_STEP` CMake option restores an `-O0` build for single-stepping.

Fault delivery to PALcode

Faults travel as a `faultCode` on the slot's `BoxResult` and are resolved at WB by `retire`. HALT is a graceful stop, not a trap: it sets `cpu.halted` and returns. Any other non-zero fault is delivered to PALcode by the sequence in the EV6 HRM (6.8.1): save the faulting PC in `EXC_ADDR` (bit 0 carries the pre-trap PALmode so `HW_REI` restores it on return), map the fault to an entry-vector offset with `coreLib::ev6::entryForFault`, divert PC to `palBase + offset`, and enter PAL mode. The mapping covers the architectural classes -- for example UNALIGN at 0x280, OPCDEC at 0x400, DFAULT at 0x380, DTB single-miss at 0x300, ITB miss at 0x580, MCHK at 0x500 -- on the 0x80-spaced vector grid from 0x100 to 0x780. The absolute target is `computeHwExceptionEntry(palBase, offset)`, which ORs the offset into the 32K-aligned `palBase`.

Delivery falls back to halt-with-diagnostic only when PALcode is not loaded (`palBase == 0`, where diverting would land in low-memory garbage and lose the failure) or when the fault has no architectural entry (`kEntry_None`). Routine high-volume TB misses (`kFaultDtbMiss / kFaultItbMiss`) are excluded from the fault log; everything else is recorded to `logs/faults.log` for offline review.

Feeding the boot path

During the firmware's self-decompression loops the IF stage consults an optional `IFetchOverride` before the normal translate-and-read, so fetches can be served from an immutable SRM payload while the guest rewrites memory underneath itself. The cold-boot inflate is the dominant early cost (see Decompressor Internals). Beyond decompression, the current runtime blocker is downstream: a clock-interrupt return through `hw_ret R2` (the genuine instruction at PC 0xd954 confirmed by the host decompressor oracle), tracked on the Current Development Status page.

See Also.

[Execution Flight Plan](#) ⁵²

[Memory Map & IPRs](#) ⁸³

[Current Development Status](#) ¹¹

Clock Frequency and the MHz Reading

12 Tools

The external tools used to build, profile, reverse-engineer, and document EmulatR. None of these are required to run the emulator; they are the developer-side toolchain behind the project.

In this section

- *Ghidra -- reverse-engineering the decompressed SRM firmware.*
- *Help & Manual -- the authoring tool that produces this documentation.*
- *Software Verify -- the Performance Validator profiler used to find hot spots.*
- *Qt Creator -- the IDE and CMake build environment.*

See Also.

[Required Tools & Frameworks](#)¹³

[Building from Source](#)¹⁴

12.1 Ghidra

Ghidra is the NSA's open-source software reverse-engineering (SRE) suite. In EmulatR it is the tool used to read the SRM firmware: once the compressed image is decompressed to a flat binary, Ghidra disassembles and decompiles it to C so that firmware addresses, functions, and behaviors can be identified by name rather than by raw PC.

Workflow

The input is the decompressed firmware produced by the host decompressor (for the DS10, `decompressed_ds10_v7_3.bin`). Analysis itself is done interactively in the Ghidra GUI -- import the binary, let auto-analysis finish, then save. The repetitive part is automated: `tools/decompile_ds10.sh` drives `analyzeHeadless` against the already-analyzed program (with `-noanalysis`) and runs the `DecompileToDir.java` post-script to dump C source plus an index file. Because headless needs the project lock, the GUI project must be closed and saved first.

Configuration notes

The project layout is fixed in the script: the Ghidra project lives at `EmulatRAppUniV4/ghidra`, the program is `decompressed_ds10_v7_3.bin`, and decompiled output is written to `tools/host_decompressor/out/decompiled_src`. One pitfall is pinned in the script: Ghidra 11.3.x must run on JDK 21. An early headless run on JDK 26 failed with `osgi.ee=UNKNOWN` because Ghidra's script compiler could not resolve the Java execution environment and never compiled the post-script, so `JAVA_HOME` is set to a JDK 21 in the script.

This is how firmware addresses cited elsewhere in the documentation -- the decompression spin, the `decompress()` entry, the `hw_ret R2 at PC 0xd954` -- were located and confirmed against real decompiled C.

See Also.

[Decompressor Internals](#)  87

[Required Tools & Frameworks](#)  13

12.2 Help & Manual

Help & Manual (EC Software) is the documentation authoring tool that produces this help project. Every page you are reading is a Help & Manual topic; the project organizes those topics into the table of contents and compiles them to the published output.

How this documentation is structured

The project is one topic per subject. Each topic is an XML file with a title, a keyword index, and a body; the table of contents is a separate map that nests the topics into the eight-section structure. Pages cross-reference each other through topic links rather than copied text, so a fact lives on one page and is linked from the others.

Authoring conventions

A consistent house style keeps the project uniform: body text uses the Normal style, section headings use the Heading styles, console and code text uses the Code Example style, and a See Also block of topic links closes most pages. Two rules matter for searchability and portability -- the keyword index on each page names every noun on that page, and the text is ASCII-only (a double hyphen stands in for a dash). These are the same conventions the rest of this documentation follows.

See Also.

[Attributions & Credits](#)  59

[Required Tools & Frameworks](#)  13

12.3 Software Verify

Software Verify (Software Verify Limited) makes a family of C/C++ analysis tools for Windows; the one used in EmulatR development is Performance Validator, a sampling and instrumenting profiler. It is the tool used to find where the emulator spends its time so optimization effort lands on the code that matters.

What it provides

Performance Validator attributes run time to functions and lines and presents call graphs and call trees, so a hot path is visible rather than guessed. For an instruction-set emulator this is essential: a small number of routines dominate execution, and a few percent shaved off the inner loop is worth more than any amount of tuning elsewhere.

How it is used here

Profiling confirmed that the pipeline driver's per-instruction `step()` is the overwhelming majority of run time (on the order of 82 percent), which is why that loop ships optimized and the -O0 debug build is gated behind a separate CMake option. It also quantified the cold-boot decompression spin and the bounded firmware wait loops -- the measurements behind the performance items elsewhere in this documentation. The profiler is a developer tool only; it is not involved in running the emulator.

See Also.

[Execution / Boot Internals](#) ⁸⁹

[CMake Macro Guards](#) ⁷⁶

12.4 Qt Creator

Qt Creator is the IDE used to develop and build EmulatR. It drives the CMake build, manages the compiler kit, and provides the editor and debugger; the project's reference build kit is Qt 6.10.2 with the MSVC 2022 64-bit toolchain.

Build environment

EmulatR builds with CMake and a C++20 MSVC compiler. Qt Creator configures the CMake project against the selected kit and produces the usual Debug and RelWithDebInfo build trees. The emulator core is deliberately Qt-free -- Qt is used only by the console / UI subsystem -- so the dependency on Qt is confined to the front end even though the whole project is built through the Qt Creator environment.

Build options that matter

Several behaviors are CMake options surfaced as build configuration rather than runtime flags. The most notable is `EMULATR_DEBUG_STEP`, which restores an -O0 build of the per-instruction step for single-step debugging; it is off by default so the hot loop ships optimized. Other compile-time guards (trace hooks, diagnostic channels) follow the same pattern and are described under CMake Macro Guards.

See Also.

[Building from Source](#) ¹⁴

[CMake Macro Guards](#) ⁷⁶

[Required Tools & Frameworks](#) ¹³

12.5 Host Decompressor

`tools/host_decompressor` is a native, host-compiled build of the original DEC SRM Huffman / inflate decompressor. It serves as a trusted reference -- an oracle -- for the compressed Alpha firmware images, producing the decompressed firmware byte-for-byte

without running any emulation. This page is the tool view (build, run, outputs); the algorithm and its place in the boot path are covered under Decompressor Internals.

Why it exists

The guest firmware self-decompresses on the emulated CPU at every cold boot (about 4 million cycles -- the spin at PC `0x60111c`). To confirm that EmulatR's CPU produces the correct decompressed image without trusting either EmulatR's own CPU or AXPBox, the tool builds the same algorithm DEC used, from the same sources the firmware was compiled from, as a native host program. The reference image it produces matches EmulatR's emulated-CPU output byte for byte, which makes it both a regression guard and a clean way to mint a decompressed ROM. It is strictly stronger as an oracle than AXPBox's `decompressed.rom`, which is generated by running the guest decompressor on AXPBox's own emulated CPU and so still depends on a correct CPU and a contrived entry sequence.

Sources

Under `src/`, pinned copies of the read-only DEC originals plus the harness:

- *`inflate.c` -- Mark Adler `inflate`, version `c10p1` (the engine).*
- *`decom.c` -- the DEC wrapper (main + decompress); reference only, not compiled.*
- *`decomp.h` -- types and the ROM / non-ROM switches.*
- *`ev6_huf_decom.m64` -- the guest assembly startup that calls `decompress()`; reference only (it also performs CPU / PAL state setup, which any in-emulator host path must preserve).*
- *`oracle.c` -- the harness: it locates the WimC header, sets the input / output pointers and compressed size, calls `inflate()`, and dumps the image.*

Build and run

Build with the bundled script (`build.sh` on Linux / `gcc, build.bat` on Windows), then run the oracle against a compressed firmware image:

```
sh build.sh
./oracle <path-to-compressed-firmware.exe>
```

Output is written to `ref_decompressed.bin` with a printed summary; a known-good run of `es45_v7_3.exe` is cached under `out/`. The `build_firmware_variants.sh` helper drives the oracle across multiple firmware images (the DS10 image, for example, yields `decompressed_ds10_v7_3.bin`, which is the input the Ghidra workflow analyzes).

Compressed-image layout

The oracle finds the firmware payload by a linear scan for the WimC magic (`0x436D6957`). For `es45_v7_3.exe` the header sits at file offset `0x2400`; its longwords give the compressed size (`0x1a6164`), the decompress target (`0x8000`), and the start of the compressed data (WimC + 20 = file `0x2414`). The decompressed output is `0x30cc00` bytes (3,197,952).

The regression guard

In the decompressed image the V4-runtime quad at PA 0xd950 is 0x7be2a000_b4de0030 -- the `hw_ret R2` at 0xd954 uses R2 (byte 0xe2), never R6 (0xe6). This is the signature that settled the 2026-05-30 single-bit-PAL-corruption question: that instruction is genuine firmware, not a decompression artifact. If a future change makes the guest CPU produce R6 in that neighborhood, it is a CPU regression, not a fix.

Integrating the oracle as a runtime alternate decompression path (`--decompress=inline|host|cache`) is deferred until the SRM reaches the `>>>` prompt; the recommended intercept-hybrid design is described under Decompressor Internals.

See Also.

[Decompressor Internals](#)  87

[Ghidra](#)  91

[Firmware Decode & Decompression](#)  51

Index

- . -

.axpsnap 10, 39, 40

- 0 -

0x1F0 37

- 1 -

10023 21, 24

1024 MB 32

- 2 -

21264 6, 46, 67, 72, 79

21264A 6, 72

21272 6, 10, 32, 46, 67, 79

21272 Tsunami 26

21272-CA 6, 9

21272-DA 6, 9

21272-EA 6, 9

268 MHz 53

- A -

AAR 50

Alpha AXP 6, 46, 57, 79

Alpha EV6 9

AlphaServer 6, 67

AlphaServer DS10 9, 67

analyzeHeadless 91

appendices 66

arbiter 9, 50

architectural correctness 8

architecture 46

ASM 85

ASN 50, 85

ATA disk 34

ATA fixed disk 36

ATAPI 6, 10, 32, 34, 36, 38

ATAPI signature 37

attributions 55, 59

authoring tool 92

auto_action 21, 33, 42, 63

autoload-newest 40

--autosnapshot 16, 17, 25, 39, 40

AXPBox 93

- B -

b dqa1 11, 63

background process 17

BAR 67

bare image 34

bash 17

Bcache 9

Berkeley SoftFloat 73

blank-4G 34

boot 30, 31, 33, 42

boot milestone 11

BOOT NOT POSSIBLE 63

boot problems 61

boot transcript 26

boot_osflags 33, 42

bootdef_dev 33, 42

Box 8, 57

BoxResult 46, 79, 89

build 13, 93

build prerequisites 13

build.sh 93

- C -

C++20 6, 13, 58

calibration 53

call graph 92

CALL_PAL 83

CALL_PAL HALT 22

canonical VA 85

CBox 46, 79

Cchip 6, 9, 32, 46, 50, 79

CD001 37

cfenv 73

clean shutdown 22, 68

CLRFEN 73

CMake 13, 14, 58, 93

CMake guards 73

CMake option 76

codegen 73, 75
 cold boot 16, 25, 51, 52
 COM1 6, 9, 33, 69
 COM2 6, 9, 69
 command line 17
 commercial license 7
 commit point 8
 compile guard 76
 compressed firmware 87
 concurrent instances 23, 25
 configure 14
 configuring 23
 connection refused 29, 62
 console 21, 23, 28, 30, 61
 console log 17
 console port 29, 61
 console server 29
 conversational boot 42
 copyright 7
 core logic 32
 CPU pipeline 10
 cpuCount 24
 CpuState 46, 79
 create_if_missing 34
 credits 59
 current directory 17
 CY82C693 10, 34, 37, 38
 CY82C693 IDE 32
 cycle counter 53
 cycle_count_freq 53
 Cypress 82C693 9, 11, 26, 32
 Cypress CY82C693 6, 46, 67, 69, 79

- D -

d command 64
 d pmem 63
 Dchip 6, 9, 46, 79
 DE500 67
 DEC 59
 DEC Alpha 6
 dec_disk_media_types 34
 decode 75
 decom.c 87, 93
 decompile_ds10.sh 91
 decompiler 91
 DecompileToDir.java 91
 decompress 51

decompressed_ds10_v7_3.bin 93
 decompression 46, 52
 deep-dive 66, 79
 dependencies 55
 deposit 64
 determinism 8
 developer 6
 developer lens 79
 developer reference 66
 development status 11
 development tools 91
 diagnostic build 76
 diagnostics 23, 43
 differential validation 8
 disk geometry 34
 disk_types.json 34
 disown 17
 dispatch tables 73
 DispatchKind 75
 DispatchTables.cpp 75
 Display 41
 dka 11
 dka0 42
 documentation 92
 dq_long_wait_on_busy 70
 dq_wait_for_drq 70
 dq_wait_on_busy 62, 70
 dqa 37
 dqa0 10, 26, 32, 34, 36, 37, 42, 62, 67
 dqa1 42, 67
 DS10 6, 10, 24, 67
 ds10_flash.rom 25
 ds10_platform.json 10, 24, 67
 ds10_v7_3.exe 6, 14, 17
 DS20 67, 72
 DSSI 39
 DTB 10, 50, 85
 dva0 11, 32

- E -

e command 64
 e pmem 63
 EBox 46, 79
 EmulatorSettings 24
 EMULATR 6, 43
 Emulatr root 15
 EMULATR VIRTUAL CDROM 37

- Emulatr.exe 14, 16
 - EMULATR_ 33
 - EMULATR_AUTOSNAP 40, 55
 - EMULATR_BOOTSTRAP_ITB_BYPASS 76
 - EMULATR_BREAK_ON_PA10 44
 - EMULATR_CHECKPOINTS 44
 - EMULATR_CHIPSET_DIAG 76
 - EMULATR_CHIPSET_DIAG_OFF 44
 - EMULATR_CONSOLE_MIRROR 25
 - EMULATR_CONSOLE_PORT 24, 25, 29, 55, 61, 68
 - EMULATR_CONSOLE_SNAPSHOT 25, 40, 55
 - EMULATR_CONSOLE_TRACE 13
 - EMULATR_DEBUG_STEP 76, 93
 - EMULATR_DIAGNOSTIC_LOGGING 76
 - EMULATR_FLASH_ROM 25, 55, 61, 68
 - EMULATR_FLASH_TRACE 25, 44
 - EMULATR_FP_SOFTFLOAT 73, 76
 - EMULATR_GATE 44
 - EMULATR_GCT_WATCH 44
 - EMULATR_IDE_TRACE 25
 - EMULATR_IIC_TRACE 25, 44
 - EMULATR_MEMDIAG 14, 44, 55, 76
 - EMULATR_NO_AUTOLOAD 16, 25, 40, 55
 - EMULATR_NO_PUTTY 15, 21, 29, 55
 - EMULATR_PA_DUMP 76
 - EMULATR_PIC_TRACE 44
 - EMULATR_PLATFORM_CONFIG 10, 24, 55
 - EMULATR_PROFILE_ALPHA_CLOCK_HZ 53
 - EMULATR_RETIRE_TRACE_DIR 44
 - EMULATR_SNAPSHOT_MARKER 40
 - EMULATR_SNAPSHOT_PRUNE 40
 - EMULATR_STOP 22
 - EMULATR_STOP_FILE 55
 - EMULATR_TRACE_DIR 17
 - EMULATR_TRACE_HOOKS 76
 - EMULATR_TRACE_WINDOW 44
 - EMULATR_USE_OS_PAGES 76
 - EmulatrV4.ini 24
 - engineering status 70
 - entry vector 89
 - environment variable 68
 - environment variables 55
 - eNVy Systems 7
 - epoch invalidation 85
 - ES40 6, 9, 10, 67, 72
 - es40_platform.json 72
 - ES45 6, 9, 10, 67, 72
 - es45_platform.json 72
 - es45_v7_3.exe 93
 - EV6 6, 10, 26, 32, 46, 67, 79
 - ev6_huf_decom 51
 - ev6_huf_decom.m64 87
 - EV67 6, 10, 72
 - Ev6EntryVectors 46, 79
 - Ev6Translator 46, 50, 79, 85
 - ewa0 42
 - examine 64
 - EXC_ADDR 83, 89
 - ExecCtx 46, 79, 89
 - execution model 46
 - Exit 41
 - explicit serialization 8
- ## - F -
- F_float 73
 - faultCode 89
 - FBox 46, 73, 79
 - FDC 6
 - FDD 69
 - Feature Matrix 10
 - FEN 73
 - fidelity standard 8
 - fidelity-faithful 8
 - FileBlockMedia 36
 - firmware 6, 13, 16, 17
 - firmware 14, 16, 17, 24, 61
 - firmware integrity check 14
 - firmware path resolution 14
 - firmware update 23, 40
 - firmwareImage 14, 24
 - firmwareSha256 14, 24
 - first run 15
 - first startup 15
 - Flash 10
 - flash corruption 25
 - flash isolation 25
 - floating point 73
 - FLOPPY 9
 - forked process 17
 - fpBox 46, 79
 - fpBoxLib 73
 - FPCR 73
 - FRU 21
 - FRU EEPROM 67

full_powerup_diags 26, 33, 62

- G -

G_float 73
 g_primaryTable 75, 89
 GCT 21, 52
 GCT/FRU 62
 genGrains.py 75
 getenv 55
 Ghidra 13, 91
 glossary 55, 57
 goals 8
 graceful-stop 22
 grain 46, 57, 75, 79, 89
 grain factory 73
 GrainMasterV4 46, 79
 GrainMasterV4.tsv 75
 GrainStubs.cpp 75
 GuestMemory 46, 50, 79

- H -

HALT 10, 21, 31, 33
 halt button 61, 63
 halt gate 63
 halt switch 63
 handwritten.tsv 75
 HBA 38
 headless 91
 help 31
 Help & Manual 91, 92
 help project 92
 host_decompressor 51, 87, 93
 HostBackend 73
 hot path 92
 Huffman 51
 HW_IPR 83
 HW_MFPR 46, 79, 83
 HW_MTPR 46, 79, 83
 HW_REI 89
 HW_RET 52, 87
 HWRPB 53

- I -

I_CTL 83

IBox 46, 79
 IDE 6, 9, 10, 34, 38, 91, 93
 IDE controller 37
 IDENTIFY DEVICE 37
 IDENTIFY PACKET DEVICE 37
 IEEE S 73
 IEEE T 73
 IFetchOverride 89
 IFpBackend 73
 IIC 10, 67
 inflate 51, 87
 inflate.c 93
 ini default 17
 IniLoader 24
 Installation 6, 13
 intercept-hybrid 87
 internal processor register 83
 interval timer 53
 intrclock_freq 53
 Introduction 6
 IPR 46, 57, 79, 83
 ISA 9, 32
 ISA table corrupt 63
 ISO 36
 ISO-9660 37
 ITB 10, 50, 85

- J -

JDK 21 91

- K -

KBD 9
 kCchipTimerBit 53
 keywords 92
 kill EmulatR 68
 kit 93
 known limitations 68
 km\$_sleep 62
 kseg 46, 50, 57, 79, 85

- L -

Launching EmulatR 15
 LBA 16 37
 leaf 75

leafBase 75
legacy interconnect 39
LFU 11, 21, 26, 40
license 7
license header 7
licensing 59
List 41
listener 62
localhost 21
LockMonitor 46, 50, 79
--log-file 17
logging 43
logical device 69
LPT1 6, 9, 69

- M -

Machine 46, 79
machine state 39
Mark Adler 51, 59, 87, 93
--max-cycles 16, 17
MBox 46, 79
media 36
MEDIUM NOT PRESENT 37
--mem 17, 32
MemDrainer 89
memory 46
memory array 32
memory map 79
memory sizing 21
memory_test 26, 33, 62
memorySize 32
MHz 53
mkdisk 34
mmap 50
MMIO 50
MobaXterm 28, 29
model 24
MOUSE 9
MSVC 13, 14, 58
MSVC 2022 93
multiple instances 29

- N -

netcat 28, 29
netstat 29

--no-autoload 16, 17, 40
nohup 17, 25
non-commercial 7
nt_fw 41
NVRAM 10, 11, 25, 30, 31, 33, 68
NVRAM flush 22
NVRAM slots 33
NXM 50

- O -

objectives 8
oem_string 33
opcode 75
OpenVMS 6
OpenVMS boot 42
operating 23
operator 6
option firmware 36
option-firmware scan 62, 70
oracle 51, 73, 87, 93
oracle.c 93
OS boot 11

- P -

PACKET command 37
PAL_BASE 51, 52, 83
PAL_TEMP 83
PalBox 46, 79
PALcode 6, 8, 10, 26, 31, 46, 57, 79
PALcode trap delivery 89
PALcode-visible IPR 83
PCC 53
PCF8584 10
Pchip 6, 9, 46, 79
PCI config 37
PCI device manifest 67
PCI enumeration 11
PCI hose 9, 32, 38
PCI probe 21
PCI-to-ISA bridge 6
performance 23
Performance Validator 44, 92
per-model reference 67
personality 75
physical address 64

physical address map 83
 Pic8259Pair 46, 79
 pipeline 79
 PipelineDriver 44, 46, 79, 89
 PipelineSlot 46, 79, 89
 planned platform 72
 platform appendix 66, 67
 platform manifest 9, 10, 23, 61, 67, 72
 platform selection 10
 PlatformConfig 24
 platformConfigPath 10, 24
 Plink 6, 13, 21, 28, 29
 pmem 63, 64
 port 24, 62
 port 10023 15, 28, 29
 port in use 61
 power-up 21, 52
 power-up sequence 26
 precedence 24
 preset 14
 primary channel 37
 primary table 75
 profile clock 53
 profile file 44
 profiler 43, 91, 92
 profiling 92
 PT0 83
 PTE 50
 PuTTY 6, 10, 13, 21, 28, 29
 PuTTY auto-launch 15, 29
 puttyPath 24
 PVD 37
 Python 58

- Q -

Qt 13, 58
 Qt 6.10.2 93
 Qt Creator 91, 93
 Quick Start 6, 16

- R -

RA disk 39
 raw image 36
 RAW mode 62
 RAW TCP 21, 28, 29

RAW TCP socket 29
 RAW-TCP client 13
 READ(10) 37
 ref_decompressed.bin 93
 reference 55
 register level 79
 regression guard 93
 release 14
 relocation 52
 RelWithDebInfo 14
 remote host 29
 REQUEST SENSE 37
 RESTART 33
 retire 89
 retire-PC histogram 44
 RetireProfiler 44
 reverse engineering 91
 roadmap 11
 ROM 14, 24
 rounding mode 73
 RPCC 53
 run directory 17
 run.log 16
 RUN/STOP 36, 63
 running 23
 RZ28 34, 38
 RZ29 38

- S -

S_PalLinux 75
 S_PalTru64 75
 S_PalVms 75
 scbd field 83
 SCSI 10, 34, 38
 scsi_poll 38
 scsi_reset 38
 SDI 39
 secondary table 75
 sector 36
 SemanticFlags.tsv 75
 separation of concerns 8
 set 30, 31, 33
 show 30, 31, 33
 show config 26, 31, 32
 show device 26, 31, 32
 show memory 31, 32
 SIO-TRACE 69

six-stage pipeline 89
 SMP 8
 snapshot 10, 11, 16, 39
 --snapshot-on-pc 40
 snapshots 23, 25
 snapshots directory 40
 SoftFloat 58, 59, 73
 SoftFloatBackend 73
 Software Verify 91, 92
 SPAM 46, 50, 57, 79
 SPAMShardManager 46, 50, 79, 85
 sparse image 34
 sparse paging 50
 spdlog 13, 43, 44, 58
 SRE 91
 SRM 6, 10, 13, 30, 46, 52, 57, 79
 SRM boot 69
 SRM commands 31, 69
 SRM console 6, 8, 23, 31
 SRM firmware 91
 SRM firmware image 14
 SRM image 17
 SRM prompt 11, 15, 16, 21
 SRM V7.3 14
 SRM variables 33
 srm_fw 41
 SRMConsole 24
 SROM 11, 51, 52, 57
 SROM parameters 87
 ST506 39
 step loop 92
 stop sentinel 22
 storage 23, 34
 storage fabric 38
 styleclass 92
 subDecode 75
 subsystem internals 66, 73
 SuperIO 6, 10, 69
 superpage 50, 85
 Supported Systems 9
 switches 17
 sys_serial_num 63
 SYSBOOT 42
 system serial number 63

- T -

table of contents 92

task matrix 70
 taskkill 22
 TBIA 85
 TBIAP 85
 TBIS 85
 TCP console 6, 10, 16, 21, 62
 tee 17
 telnet 28, 29
 Tera Term 28, 29
 terminal client 13, 28
 TIG 9
 timeout 62
 timeout loop 62, 70
 Timothy Peer 7
 TLB 10, 46, 50, 79
 TlbEpoch 46, 50, 79, 85
 toolchain 93
 tools 13
 topic 92
 topiclink 92
 trace 43
 --trace 17
 traced boot 25
 translation 46
 translation buffer 85
 trap mode 73
 troubleshooting 29, 61
 Tru64 boot 42
 Tru64 UNIX 6
 Tsunami 6, 9, 57, 67
 Tsunami21272_CsrSpec 53
 TsunamiChipset 46, 50, 79
 tulip 67, 70
 two instances 25
 Typhoon 6, 9, 46, 79

- U -

UPD 21, 26, 40, 41
 Update 41
 update srm 26, 40, 41
 User Guide 6

- V -

VA_CTL 83, 85
 VAX float 73

Verify 41
virtual CDROM 32, 36
virtual CD-ROM 37
virtual disk 34, 36
VirtualAlloc 46, 50, 79
VirtualIsoDevice 37
VirtualScsiDevice 38
Visual Studio 58

- W -

will not start 61
WimC 87, 93
word-fold 69
Writeback 8
wwid 33